DEEP RESEARCH · PRODUCT STRATEGY · ENGINEERING BLUEPRINT

Building the AI Workforce

Persistent, proactive multi-agent systems that perform and verify real work: teardowns of Grok Bot, Instinct, Muse and Cursor, five open-source repositories, the super-agent architecture, and a staged path from first customer to production.

Prepared for the founding and engineering team

16 deliverables · verified sources · dated November 2026

NOVEMBER 2026

Table of Contents

Executive Strategy . . . 1

1.1 The verdict in one paragraph . . . 1

1.2 Why this opportunity, why now . . . 2

1.3 Who the first customer is . . . 3

1.4 The four ambitions, applied . . . 3

1.5 What a top professional would add that this brief overlooks . . . 4

1.6 Success criteria for the whole venture . . . 4

Product Teardowns: Grok Bot, Instinct, Muse, Cursor . . . 5

2.1 Grok Bot (xAI) - the persistent AI teammate, shipped at scale . . . 5

2.2 Instinct (instinct.com) - the no-interface assistant . . . 6

2.3 Meta's Muse - identity files, memory folders, and a cautionary tale . . . 7

2.4 Cursor (Anysphere) - the engineering playbook worth copying . . . 7

2.5 The Maven course - Cursor's engineering-skills framework . . . 8

2.6 Cross-product comparison . . . 8

2.7 Adopt, improve, avoid . . . 9

Repository Teardown and Reuse Assessment . . . 10

3.1 CopilotKit/OpenDots - the cleanest product-shape reference . . . 10

3.2 elie222/rakazo - the most complete open AI-workforce stack . . . 11

3.3 OpenHands - the coding-agent engine, now 'Agent Canvas' . . . 12

3.4 fayazara/coworkers - the serverless architecture sketch . . . 12

3.5 twentyhq/twenty - the CRM foundation question . . . 13

3.6 Build-versus-integrate decision . . . 13

Product and UI/UX Blueprint . . . 15

4.1 Screen inventory and purpose . . . 15

4.2 Design system . . . 16

4.3 Component inventory . . . 16

4.4 Critical journeys, end to end . . . 17

4.5 Confusing flows found in the wild, redesigned . . . 18

Agent Avatars and Genuine Behavior . . . 19

5.1 The operational state model . . . 19
5.2 How progress, logs, and results are communicated . . . 20
5.3 Never fake activity - the integrity rules . . . 21
The Agent Harness . . . 21
6.1 The complete lifecycle . . . 22
6.2 What the harness provides, concretely . . . 23
6.3 Chatbots, tool-callers, task agents, persistent systems . . . 24
6.4 The autonomy ladder . . . 24
Super-Agent Architecture: Router, Workers, Events . . . 25
7.1 Mechanisms A-G, verified and evaluated . . . 25
7.2 What the experiments actually showed . . . 27
7.3 Scaling, limits, and what we take . . . 27
Skills, Browser, and the Multi-Agent Workforce . . . 29
8.1 The skills registry . . . 29
8.2 The browser and internet stack . . . 29
8.3 The specialist workforce and supervision . . . 30
Agentic Engineering and Self-Improvement . . . 31
9.1 The engineering workflow agents should run . . . 31
9.2 What the research actually supports . . . 31
9.3 The safe improvement loop . . . 33
Real-World Problem Selection . . . 33
10.1 Candidates evaluated . . . 34
10.2 The recommended wedge and the industry backdrop . . . 35
Technical Architecture . . . 35
11.1 Stack decisions with rejected alternatives . . . 35
11.2 System architecture . . . 37
11.3 Data model . . . 38
11.4 API and event contracts . . . 38
11.5 Repository structure and deployment . . . 39
11.6 Security and reliability hard rules . . . 41
Monitoring, Reliability, and Validation . . . 41
12.1 What we measure . . . 41

12.2 The Watchdog as a reliability component . . . 42
12.3 Evaluation that cannot grade its own homework . . . 43
Risks, Costs, and the Phased Roadmap . . . 43
13.1 Stages . . . 43
13.2 What we deliberately do not build (yet) . . . 44
13.3 Platform-level pros, cons, and risks . . . 45
Worked Example: The Competitive-Intelligence Agent . . . 46
14.1 Configuration . . . 46
14.2 Independent verification protocol . . . 47
Final Verdict and the First Build . . . 48

Executive Strategy

This report answers one question: **what should a small team actually build** if it wants a real AI workforce - persistent, proactive, multi-agent systems that perform and verify useful work - rather than another chatbot or a dashboard with animated avatars. The research underneath is empirical. Between August and November 2026 we inspected the shipping products in this category (xAI's Grok Bot, Instinct, Meta's Muse, Cursor's agentic tooling), pulled the source code and licenses of five open-source repositories (CopilotKit/OpenDots, elie222/rakazo, OpenHands/OpenHands, fayazara/coworkers, twentyhq/twenty), read the November 2026 DeepSeek Harness super-agent design article in full, and reviewed the primary academic literature on self-improving and long-horizon agents. Every factual claim is labeled and dated; the evidence classes are kept separate: **verified** (we fetched it), **official claim** (the vendor says it), and **inference** (our judgment). Where evidence was unavailable - pricing pages behind bot-walls, install counts not exposed - we say so instead of guessing.

1.1 The verdict in one paragraph

Build a **vertical AI-workforce product for competitive and market intelligence**, sold to B2B SaaS marketing and strategy teams, in which named, persistent agents run scheduled research workflows in sandboxes, attach verifiable citations and archived evidence to every claim, and publish an audited weekly intelligence brief. Do not build a horizontal agent platform first. The horizontal layer - agent runtimes, marketplaces, bot-to-bot messaging - is being commoditized at astonishing speed by xAI, Meta, Microsoft, and an aggressive open-source ecosystem (rakazo shipped a self-hosted Grok Bot alternative with seven sandbox providers in under three months). The durable gap, confirmed by both the funding record and the failure literature, is **trustworthy execution**: Gartner projects over 40% of agentic AI projects will be canceled by end-2027 (cited by Forbes, Jul 2026), while Sierra raised $950M at a $15B valuation (May 2026) selling exactly the thing that is scarce - reliable, auditable agent outcomes. Verification is the product.

1.2 Why this opportunity, why now

Three facts make the timing concrete. First, demand evidence: Clay - the closest incumbent in go-to-market data and research automation - raised $115M at a $7.1B valuation in September 2026 and prices teams at $167-$446 per month for action-based research work (clay.com/pricing, Nov 2026), while dedicated page-monitoring tools like Visualping sell for $14-$140 per month (visualping.io/pricing, Nov 2026). Between those two anchors sits an underserved middle: teams that need analyst-grade, cited intelligence briefs, not raw page diffs. Second, capability evidence: coding agents resolved 74-79% of SWE-bench Verified by late 2025 (swebench.com, Feb 2026 snapshot), GAIA's public leaderboard crossed the 92% human baseline in 2026, and METR measures frontier-agent task horizons doubling roughly every seven months (arXiv:2503.14499). Third, pattern evidence: Grok Bot launched August 11, 2026 with a marketplace of 87+ community agent templates and 56 documented use cases (x.ai, Nov 2026) - proof that the 'AI teammate with a job description' mental model is already being taught to millions of users by better-funded players. A small team cannot win the platform war; it can win a workflow.

Bet	What it means in the product	Evidence that it is right
Verification-first harness	Every task ends in machine-checkable proof: re-fetched citations, archived snapshots, test runs, diffs. Agents never self-certify.	OpenAI retired SWE-bench Verified over contamination (Apr 2026, HN 343 pts); 'Do Agent Optimizers Compound?' shows uncontrolled self-improvement inverts under shift (arXiv:2607.14004).
Persistent named agents	Agents are long-lived identities with versioned charters, memory, and skills - not per-chat prompts.	Grok Bot's 3-field bot definition + skills/memories/routines schema (x.ai, Nov 2026); Muse's SOUL.md/ IDENTITY.md/MEMORY.md runtime files (mouse.dev teardown, Sep 2026).
Event-driven super-agent core	Triggers from email, Slack, GitHub, and schedules pass a deterministic Fast Path before any model call; a Router owns threads; isolated Workers execute.	DeepSeek Harness article: two-level architecture beat single-session delegation p95 15.7 vs 29.5 min; pass-through cut Router cache-read tokens 37% (Nov 2026, verified text).
Vertical GTM motion	Sell the weekly audited brief outcome, not 'agents'. Land $500-$2,000/mo team plans.	Clay $7.1B valuation; Sierra/Decagon pricing power in vertical agents; Crayon/Kompyte sales-gated legacy pricing leaves the mid-market open.

Table 1.1 - The strategic bets and their evidence

1.3 Who the first customer is

The initial customer is a **50-500 person B2B SaaS company** with a product-marketing or competitive-intelligence function that today runs on a patchwork: Google Alerts, Visualping-style diffing, manual spreadsheet monitoring, and an analyst spending 5-10 hours a week compiling summaries. The pain is specific and measurable - competitors ship pricing changes, feature launches, and messaging shifts that are noticed late or not at all, and existing tools either flood the team with raw diffs (Visualping at $14/month) or demand enterprise sales cycles (Crayon, Kompyte). The buyer has budget authority, the deliverable is a document a VP can read in five minutes, and success is independently checkable - every claim in the brief either traces to a fetchable, archived source or it does not. Agencies that resell competitive research are the second beachhead: Clay's own packaging targets them, which validates willingness to pay.

1.4 The four ambitions, applied

Altitude. The ultimate outcome is not 'agents that chat' but an organization's ambient research department: it watches, it verifies, it reports, it remembers why. Beyond the chat interface, the product's real surface is the inbox and Slack thread where a calm, cited brief appears every Monday morning - and an audit trail a compliance officer can open a year later.

Depth. We inspected raw source, not READMEs: rakazo's sandbox adapter layer (packages/adapters with Docker, E2B, Daytona, Kubernetes clients), OpenHands' migration to the Agent Server + SDK architecture with 907 test files, Twenty's 58 CI workflows and merge queue, the Grok Bot marketplace payload's actual bot record schema. The engineering conclusions in chapters 3, 7 and 11 come from that level.

Range. The design deliberately fuses product strategy (chapter 10), UX/HCI (chapters 4-5, where fake-activity animations are treated as an integrity problem), AI engineering (chapters 6, 8), distributed systems (chapter 7's Router/Worker analysis), security (chapter 11.8 - prompt injection is treated as unsolved and designed around), and unit economics (chapters 13-14). Each discipline changes a concrete decision, not just the prose.

Speed. The MVP reuses aggressively: container sandboxing patterns from OpenHands/rakazo (MIT/Apache-2.0), the skills-as-markdown convention converging across Cursor's pstack, the Agent Skills standard donated to the Linux Foundation's AAIF (Oct 2026), and LangGraph's checkpoint/HITL concepts - while writing the thin orchestration core ourselves so the audit log is ours. Chapter 13 stages the build so that a 3-6 person team reaches a paying pilot in roughly one quarter without Kubernetes, microservices, or a bespoke model.

1.5 What a top professional would add that this brief overlooks

Four things, all incorporated. **(1) Sell before building:** Stage 0 is a concierge pilot - run the workflow manually-with-models for 3 design partners at $500/month before the platform exists; this costs nothing and tests willingness to pay, which no amount of architecture de-risks. **(2) Inspect failures, not demos:** the raw evidence that matters is failure distributions - Grok Bot's public security incidents (Techdirt, Oct 2026: a bot leaked a CEO's bank data to a shared Slack), Muse's privacy review cluster (HN, 666 pts, Sep 2026), and the SWE-bench contamination retirement - because they define the trust bar a new entrant must clear. **(3) Instrument economics from day one:** cost per successful task is a first-class metric (chapter 12), because at $0.02-$0.05 per research run in model spend, the difference between a 3-step and a 30-step agent is the whole margin. **(4) Prove it works independently:** chapter 14 defines the external verification protocol - a citation re-fetcher that is not the authoring agent, a human spot-check sampler, and an 8-week benchmark against the incumbent manual process.

1.6 Success criteria for the whole venture

We commit to numbers now so the roadmap can be judged against them. Pilot gate (end of Stage 1): 3 paying design partners, 90%+ of weekly briefs delivered on schedule, every cited claim independently re-fetchable. Product gate (end of Stage 3): task success rate at or above 85% on a frozen 50-task benchmark suite, cost per successful brief at or below $3.00, human edits per accepted brief at or below 1 on average, week-4 retention at or above 60% of pilot tenants. Business gate (end of Stage 4): $15k MRR from 10-15 tenants, at least 2 references willing to talk, gross margin at or above 70% after model and sandbox spend. These thresholds are deliberately conservative relative to the frontier results in chapter 9, because a small team optimizes for reliability, not peak capability.

Sources

Grok Bot launch and marketplace: x.ai/news/introducing-grok-bot; x.ai/bot/marketplace/engineering (accessed 2026-11-04)

Meta Muse identification and teardown: HN item 49615537 (2026-09-08); mouse.dev/blog/muse-runtime-export (2026-09-22)

Clay funding and pricing: SiliconANGLE (2026-09-09); clay.com/pricing (2026-11-04); Visualping pricing: visualping.io/pricing (2026-11-04)

Gartner 40% cancellation projection via Forbes (2026-07-07); Sierra $950M at $15B via SiliconANGLE (2026-05-04)

SWE-bench Verified leaderboard: swebench.com (snapshot 2026-11-04); OpenAI retirement note via HN item 47910388 (2026-04)

METR time-horizon: arXiv:2503.14499 (2025-03-18)

Product Teardowns: Grok Bot, Instinct, Muse, Cursor

This chapter is an evidence-based teardown of the four shipping products the brief names, plus the Maven course that documents Cursor's internal methods. One identification note up front: 'Muse' is ambiguous in the original brief. We verified that the product almost certainly intended is **Meta's Muse** (muse.ai, launched around September 8, 2026), not the 2015-era muse.io page builder or the band: the HN launch thread drew 666 points, and third-party tooling explicitly groups 'Muse/Instinct-alike' agents together (HN, Oct 2026). Meta's own pages block automated fetching (403), so Muse facts below rest on search-index copies of official pages and independent teardowns, and are labeled accordingly. The same honesty applies to Grok Bot pricing, which x.ai blocks from automated access; the widely quoted '$120/$200 per employee per month' figures are unverified Hacker News comments and are treated as rumors.

2.1 Grok Bot (xAI) - the persistent AI teammate, shipped at scale

Grok Bot is the most complete public realization of the 'AI employee' pattern as of November 2026. The launch post (Aug 11, 2026) defines it as 'your team of always-on agents. They have their own computer, work inside tools and apps like you do, and keep working 24/7' (x.ai/news/introducing-grok-bot, verified). The unit of product is strikingly simple: a bot is three fields - name, title, description - plus instructions. Each bot runs on a persistent cloud computer with a desktop, filesystem, terminal, and apps; the guide describes handing the desktop back to the human for CAPTCHAs and 2FA, and sending secure forms for credentials (x.ai/bot/guides/grok-bot-101, Sep 2026, verified). Bots interact through three modes: chat, 'routines and triggers' (schedules, watching Slack threads or GitHub PRs), and other bots - including group chats where bots message each other and trigger each other.

The governance model is the most interesting engineering decision. Permissions are written as natural-language rules in settings, and - verified from the guide - 'a separate review agent checks proposed actions and can allow, block, or escalate to you,' with work happening 'in an isolated environment.' The docs also admit the sharp edge: logging one bot into a site makes that site reachable by every other bot on the account. Distribution is a full marketplace (x.ai/bot/marketplace, verified) with roughly 95 bot records in the embedded payload (87+ unique after deduplication), ten categories, and 'Share as Template' publishing that packages skills, memories, and plugins but excludes custom scripts. The engineering category includes an official 'SWE by Cursor' bot and an 'Engineering Lead' bot authored by Lauren Tan - a Cursor staff engineer - whose public instructions delegate to Cursor cloud agents, run a 30-minute fleet-watcher cron, and state 'The human owns every merge.' Use-case coverage is broad: 56 documented jobs across sales, marketing, support, recruiting, ops, and engineering (x.ai/bot/use-cases, verified).

87+

unique agent templates live in the Grok Bot marketplace 8 weeks after launch (x.ai payload, Nov 2026)

Unknowns and risks, honestly: pricing pages are bot-walled (403 on three attempts, Nov 2026), so unit economics are unverifiable; install counts are exposed as zero in the public payload, so traction per template is unknown; and the platform has already produced a public security incident - Techdirt reported (Oct 19, 2026) a Grok Bot spilling a tech CEO's bank data into a shared Slack. The strategic lesson is double-edged: the product model works and is popular, but the trust surface - shared computers, credential reuse, review-by-model - is exactly where a new entrant can differentiate with hard isolation and human-owned approvals.

2.2 Instinct (instinct.com) - the no-interface assistant

Instinct is a consumer 'personal assistant that understands your work and takes action across your applications and devices' (instinct.com, verified Nov 2026). Its positioning is deliberately anti-UI: 'The interface is simple: there are no new interfaces. You can text or call it. It can use a phone and computer in the same way that you do.' The privacy policy (revised Aug 26, 2026, verified) is unusually revealing about implementation: the legal entity is Spear Street Technology, Inc.; the assistant may store 'your username and password for third-party accounts so that the personal assistant can sign into these accounts on your behalf'; it may process health data; and it uses customer data 'to evaluate, fine-tune, and train the AI models' with an opt-out. A dedicated Google Workspace section commits to Limited-Use and no training on Workspace data. The JavaScript bundle lists integrations: Google Workspace, Linear, GitHub, Slack (verified from the bundle).

What Instinct demonstrates for our build is interface philosophy, not architecture: for personal-assistant work, the winning surface is the channel the user already has (SMS, calls, existing apps), not a new dashboard. It also demonstrates the trust cost of the 'store my passwords and act as me' model - The Atlantic's September 2026 piece 'It's All Fun and Games Until You Give AI Your Credit Card' (existence verified via HN; body inaccessible) captures the consumer anxiety that our enterprise motion must design away with scoped OAuth and human-owned credentials. Unknowns: funding, team, user counts, model provider, and whether execution happens in cloud VMs or on the user's Mac are all unverified; the site is a React SPA with no public subpages.

2.3 Meta's Muse - identity files, memory folders, and a cautionary tale

Muse is Meta's personal AI agent ('Meet Muse, our new AI agent that takes tasks off your plate' - ai.meta.com homepage copy via search index, Sep 2026; direct fetch blocked). It ships a Mac/mobile app, a small-business mode ('Muse for Small Business'), and 'Muse Code,' described in indexed copy as 'a fast and accessible coding agent powered by Muse models' (dev.meta.ai snippet, unverified beyond the index). The most valuable public evidence is an independent teardown (mouse.dev, Sep 22, 2026, verified) in which a researcher asked Muse for its filesystem and received a 6.8 GB export. The runtime - internally codenamed 'Hatch' - is a Linux (Ubuntu) container whose agent home contains **SOUL.md, IDENTITY.md, USER.md, MEMORY.md, AGENTS.md, TOOLS.md**, plus skills/ and memory/ directories including memory/bank/, memory/dreams/, and workspace/self_improvement/, with bubblewrap (bwrap) sandboxing and a bin/codex binary in the image.

Three lessons carry directly into our design. First, the **identity-files pattern** - a human-readable charter, user profile, and memory document - validates our decision to make agent identity versioned markdown that users can read and diff (chapter 5). Second, the reception was brutal on privacy and permissions: a Techdirt piece titled 'Meta's Muse is an adorable privacy and security dumpster fire' (Oct 2026, 392 points on HN), AppleInsider alleging it 'blatantly ignores users permissions' (Sep 2026), and Forbes reporting Amazon blocked Muse from shopping on amazon.com (Sep 2026) - all HN-index verified titles. The market is telling vendors that memory and permissions need user-visible governance, not just capability. Third, even Meta ships agents with an explicit self-improvement workspace folder, confirming that 'learning' is now a table-stakes expectation - and, per the mouse.dev report filed through Meta's bug bounty, an attack surface.

2.4 Cursor (Anysphere) - the engineering playbook worth copying

Cursor is the proof that agentic methods can be made reliable enough that professionals stake their output on them, and its methods are unusually well documented. The customization model (cursor.com/docs, verified Nov 2026) has converged on: **Rules** (persistent instructions including AGENTS.md files), **Skills** (SKILL.md files with domain knowledge, workflows, and scripts), **Subagents** (delegates that run in their own context), hooks, commands, and plugins - installed from a marketplace with team leaderboards. Cloud Agents 'run in isolated VMs in the cloud with full development environments' and can be kicked off from Slack, GitHub comments, Linear, web, and mobile; **Automations** run cloud agents on schedules or in response to GitHub/Slack/Linear/webhook events, described in plain language via a /automate skill; Bugbot reviews every PR automatically. This is precisely the trigger-to-agent pipeline the super-agent article formalizes, shipped as product.

Two documents deserve deep reading. First, the blog post 'Towards self-driving codebases' (Feb 5, 2026, verified) reports a multi-agent harness that ran continuously for one week, 'making the vast majority of the commits' to a research project without human intervention - and honestly documents what failed: a single agent with 'keep going' failed; equal-role agents coordinating through a shared state file failed (lock contention, risk-averse agents); a rigid planner-executor-judge pipeline was 'too rigid.' What worked was a

continuous executor spawning workers, aggressive context freshness (scratchpads rewritten, not appended; self-reflection reminders), full observability (all messages, actions, and command outputs logged with timestamps for replay), and hill-climbing on failure patterns. Second, **pstack** (github.com/cursor/plugins, MIT, by Lauren Tan, verified from the raw README) - the open-sourced skills framework taught in the Maven course - contains 23 playbooks under /poteto-mode, 30+ skills (/architect, /tdd, /arena for N parallel attempts, /interrogate where different models try to break a result and only proven findings survive, /correct which 'mines history for mistake classes,' and /create-verification-skill which generates a project-local verify skill with a feature map), and 24 principles including 'prove-it-works: Verify against the real artifact... not a proxy, self-report, or it compiles' and 'encode-lessons-in-structure: Encode the rule as a lint, metadata flag, runtime check, or script instead of more text.' Corporate context: Cursor was acquired by SpaceX (cursor.com/blog/joining-spacex, Aug 2026, verified), and the footers now carry AIUC-1 security certification alongside SOC 2 and ISO 27001.

2.5 The Maven course - Cursor's engineering-skills framework

The course 'How Cursor Turned AI Agents Into Better Engineers' (maven.com/p/e23d9c, verified) is a free 60-minute lesson by Lauren Tan (Member of Technical Staff, Cursor) with 4,137 enrolled students, part of the 'Become an AI Native Builder' series. Its chapter list is effectively a syllabus for agent reliability engineering, and we adopt it as internal training material: 'The Agent Trust Curve: Moving from Micromanagement to Auto-Merging PRs'; 'The Importance of Verification Skills and Feature Maps for Agents'; 'Introducing PStack: Building Skills to Prevent Agent Hallucinations'; 'Maintaining Agent Skills Using Evals and the Eval Playbook'; 'Scaling Verification: From Local Observation to Cloud Agents'; 'Refactoring Codebases and Setting Guardrails for AI Agents'; 'Managing PR Sizes and Structuring Work for AI Agents'; 'Implementing Strict CI Constraints and the Dune Architecture'; 'Token Usage, ROI, and the Cost of Agent-Optimized Codebases'; 'Empowering Product Teams with Grok Bot.' The framing 'AI agents behave like talented but forgetful new hires' is the correct mental model for our skill-registry design: skills exist to give the talented hire institutional memory. The 'Dune Architecture' is named only in the chapter title; its details are not public, and we flag that as an honest gap.

2.6 Cross-product comparison

Dimension	Grok Bot	Instinct	Meta Muse	Cursor agents
Primary users	Teams/prosumers (V)	Consumers (V)	Consumers + SMB (O)	Developers (V)
Agent creation	3-field bot + instructions; templates (V)	Conversational, implicit (V)	Conversational; identity files (U)	Rules/Skills/Subagents as files (V)
Memory	Per-bot memories list (V)	Unspecified (U)	MEMORY.md + memory/ tree (V-teardown)	Codebase rules + session context (V)

Dimension	Grok Bot	Instinct	Meta Muse	Cursor agents
Browser/computer	Own cloud computer; hands back for CAPTCHA (V)	Uses user's phone/computer (O)	Linux container + bwrap (V-teardown)	Cloud VMs + browser/desktop control (V)
Delegation	Bot-to-bot messages, group chats (V)	Single assistant (V)	Unverified (U)	Subagents in own context (V)
Background runs	Routines + triggers (V)	Implied (O)	Scheduled work (V-teardown)	Automations: cron + events (V)
Monitoring	Review agent allow/block/escalate (V)	None public (U)	Memory inspection only (U)	Full trace logging; Bugbot PR review (V)
Marketplace	87+ templates, 10 categories (V)	None (V)	None found (V-search)	Plugins marketplace (V)
Pricing	Unverified ($120-200 rumor)	Paid, unpublished (V-terms)	Unverified	Cloud-agent usage-billed (V)
Security posture	Incident reported Oct 2026 (3P)	Stores credentials (V-policy)	Criticism cluster (3P)	AIUC-1 + SOC2 + ISO (V-footer)

Table 2.1 - Capability matrix across the four products (V = verified from official pages/docs; O = official claim; U = unverified)

2.7 Adopt, improve, avoid

Adopt: the persistent-agent-with-charter model and template packaging (Grok Bot); the rules-and-skills-as-versioned-files discipline and verification-skill pattern (Cursor/pstack); the identity-files readability of Muse; the channel-first interaction philosophy of Instinct for delivery surfaces. **Improve:** isolation (one sandbox per agent per run rather than shared computers), credential handling (scoped OAuth tokens with user-owned consent instead of stored passwords), and memory governance (typed, inspectable, deletable entries rather than sprawling folders). **Avoid copying:** review-by-a-single-model as the only gate (insufficient against the prompt-injection reality documented in chapter 8), unlimited bot-to-bot autonomy without budgets, and marketplace growth before the audit trail is trustworthy - the incidents above are the cautionary evidence.

Sources

x.ai: /news/introducing-grok-bot (2026-08-11), /bot/guides/grok-bot-101 (2026-09-11), /bot/guides/templates-for-grok-bot, /bot/use-cases, /bot/marketplace + /marketplace/engineering + bot detail pages (all accessed 2026-11-04)

instinct.com: homepage, /privacy-policy (2026-08-26 rev.), /terms, JS bundle (accessed 2026-11-04)

Muse: HN item 49615537 (2026-09-08); mouse.dev/blog/muse-runtime-export (2026-09-22); HN-indexed Techdirt/AppleInsider/Forbes titles (2026-09/10)

Cursor: cursor.com/docs (+ cloud-agent, automations, bugbot, customize-cursor, grok-bot), cursor.com/blog/joining-spacex (2026-08-14), cursor.com/blog/self-driving-codebases (2026-02-05), pstack README via raw.githubusercontent.com/cursor/plugins (accessed 2026-11-04)

Maven course page: maven.com/p/e23d9c (accessed 2026-11-04)

Repository Teardown and Reuse Assessment

We inspected five repositories beyond their READMEs: folder structure, dependency manifests, licenses, CI configuration, and release cadence, fetched directly (github.com HTML, raw.githubusercontent.com manifests, and the ungh.cc metadata mirror; all accessed 2026-11-04). A finding worth stating first: **all five repositories exist**, including the three the brief hedged on. CopilotKit/OpenDots (6,335 stars, created 2026-09-29) is an alpha template for persistent AI coworkers; elie222/rakazo (6,635 stars, Apache-2.0) is a self-hosted Grok Bot alternative; OpenHands (92,537 stars, MIT) has pivoted its platform to 'Agent Canvas'; fayazara/coworkers (~350-390 stars, MIT) is a Cloudflare-native bot workforce experiment; and twentyhq/twenty (60,355 stars) is the mature open-source CRM. Star counts come from a mirror and may drift slightly from live values; that is noted where it matters.

3.1 CopilotKit/OpenDots - the cleanest product-shape reference

OpenDots describes itself as 'Your always-on AI coworkers that move between text, calls, and Slack' and, importantly, 'A template, not a hosted product,' badged Alpha (README, verified). Each 'Dot' is a persistent specialist with a name, role, instructions, and a tool allowlist; Dots can own a Docker computer provisioned through CopilotKit's OpenBot container supervisor with a persistent browser profile and files; a Tiptap-based 'Spaces' document workspace saves chat outputs after a human 'Approve and save' step; MCP tools auto-run when read-only and pause with approval cards otherwise; Composio provides connected apps (Gmail, Google Calendar); Slack presence and voice/calls are built in. The server is Hono/TypeScript with a clean module split (dot-agent, runner, headless, slack-channel, voice, computer-service, page-service, workspace, connections, learning) and 58 test files - unusually disciplined for an alpha.

Constraints that block direct reuse: it is single-owner by design (OWNER_TOKEN, no multi-tenancy), storage is SQLite, and conversations require CopilotKit Intelligence - a hosted account, a licensed self-host deployment, or a 30-day evaluation container (docs/SETUP.md, verified) - so the conversation runtime is not freely swappable. License: MIT (verified, Copyright Atai Barkai), with the runtime licensing noted above as the practical encumbrance. Verdict: **2/5 production-ready, high reference value** - copy the approval-card UX, per-Dot permission toggles, and Spaces document pattern; do not build on it.

3.2 elie222/rakazo - the most complete open AI-workforce stack

Rakazo brands itself 'Open-source Grok Bot alternative. Choose your own model and sandbox' (repo description, verified) and positions against 'Grok Bot, Meta Muse, OpenAI Dots, Instinct, OpenClaw, Hermes Agent, and Claude Duet.' Bots are continuing identities with their own conversation, memory, cron routines, and computers - shared 'Team Computers' plus isolated 'Private Computers', each a browser (per-bot X display with persistent Chrome profile), terminal, and graphical desktop; bot-to-bot delegation and short-lived subagents are supported; voice is bring-your-own (ElevenLabs/OpenAI/Cartesia/Fish Audio); approvals run through an 'approval model' in which a cheap model auto-checks proposed actions. Apps span web, Electron, iOS (App Store), and Android APK. The adapter layer is the crown jewel: packages/adapters implements sandbox providers for Docker/dockerode, E2B desktop, Daytona, Boat, boxd, Novita, Railway, Alibaba Wuying, and Kubernetes. The agent runtime is 'Pi' (@earendil-works/pi-agent-core), a BYO-model harness with fallback chains and per-bot model selection; chat adapters cover Slack, Telegram, WhatsApp, iMessage, and Lark; integrations include Composio, Pipedream Connect, and remote MCP; persistence is Postgres + Prisma with row-level security, Better Auth for authn, Graphile Worker for jobs. Engineering hygiene is serious: v0.4.0 released 2026-11-01, published multi-arch Docker images, a Railway template, production self-host docs (Caddy, backups, audit log, observability), and 15 GitHub workflows including Playwright e2e, nightly verification, and computer-replay tests.

License: Apache-2.0 (verified in LICENSE and package.json) - permissive with patent grant; the only string noted is a third-party integration (Treg) whose service terms require a written agreement for hosted resale. Gaps: no CRM/business-data layer; the project's own docs concede workspace folders 'are not security boundaries' for Team bots; and it is operator-oriented (self-host), not a multi-tenant SaaS. Verdict: **3.5/5, the best integration-or-reference candidate** for our execution and computer layer; its docs are effectively a free design review for our Stage 2-3.

3.3 OpenHands - the coding-agent engine, now 'Agent Canvas'

The OpenHands repo (formerly OpenDevin) has pivoted: main is 'Agent Canvas v1.34.0' (@openhands/agent-canvas), 'the self-hosted developer control center for coding agents and automations,' which runs any ACP-compatible agent (OpenHands, Claude Code, Codex, Gemini CLI, Pi, OpenCode) across local/Docker/VM/cloud backends with conversations, terminal, browser, files, and automations (scheduled or webhook-triggered; Slack/GitHub/Linear/Datadog integrations) - README and docs/architecture.md, verified. The agent engine lives in OpenHands/software-agent-sdk (Python: openhands-agent-server with docker_runtime, persistence, storage, telemetry, and openai modules; openhands-sdk, openhands-tools, openhands-workspace; TS clients; 907 test files). The build system exposes an embeddable frontend library (browser, conversation, files, settings, terminal entrypoints). Quality signals: 22 workflows (CI, mock-LLM Playwright e2e, Docker publish, npm trusted publishing with provenance, Helm chart, skill-isolation verification), Stryker mutation testing, release-please. The SDK README badges SWEBench 77.6 - we could not verify whether that figure is SWE-bench Verified (the linked sheet was not fetched), so we report it as claimed. License: MIT throughout (verified). Verdict: **4/5 for the coding-agent substrate**; for our purposes it is a strong candidate to embed as the engineering-agent execution service (Stage 5) rather than the core runtime for research workloads, and its Automation Server pattern validates our trigger design.

3.4 fayazara/coworkers - the serverless architecture sketch

Coworkers ('AI teammates that each get their own computer and browser. Built entirely on Cloudflare', README, verified) is an experimental implementation whose value is architectural: everything is Durable Objects - one UserHub per user (bot list, connections, shared memory, installed skills, push subscriptions) and one BotAgent per bot holding its own DO-SQLite database ('no separate database'). Features map the product surface end to end: streaming resumable chat, a JS-scripts computer ('No container'), watched/drivable browser with human takeover ('bot hands you its browser to sign in'), per-action-type rules (allow/ask-first/block), routines with check-ins, teach-by-demo that records your browser session into a skill, digest-pinned skill installation (human-only install), spend tracking from AI Gateway logs, a 40-step loop guard, approvals with push reminders and 24-hour expiry, and background turns restricted to read-only tools. It is explicitly a prototype: README warning, preview/beta Cloudflare APIs, no CI workflows, created 2026-09-30, ~354-389 stars (mirror drift). License MIT (verified, 'Fayaz Ahmed' - the brief's attribution of another name is unconfirmed). Verdict: **2/5, copy the patterns** - one-store-per-agent state, digest-pinned skills, takeover flow, spend metering - not the code, which is Cloudflare-locked.

3.5 twentyhq/twenty - the CRM foundation question

Twenty is the most production-grade of the five: 'The #1 Open-Source CRM... The open alternative to Salesforce, designed for AI' (README, verified), with four years of history, 58 CI workflows including a merge queue and cross-version upgrade tests, Storybook visual regression, versioned releases (twenty/v2.54.0, 2026-10-30), SAML SSO in enterprise-marked code, and a metadata-driven object model where custom objects, fields, and views are data, not migrations. Architecture: NestJS 12 + GraphQL Yoga backend, TypeORM 0.3.31 (patched), PostgreSQL, Redis/BullMQ; React 19 + Apollo Client 4 + jotai front end (Recoil was replaced - our inference from the manifest); an Nx monorepo of 17 packages; REST + GraphQL + webhooks; and a real extension platform (twenty-sdk, twenty app init/publish) whose documented capabilities include 'objects, views, workflows, agents, and logic functions.' Notably, the server contains 739 agent-related paths including agent-chat-thread and pending-question schemas, and there is an official twenty-agent-skills package - Twenty already treats agents as first-class workspace citizens with role bindings (migrations show agent-to-role foreign keys).

Licensing is the decisive constraint, verified from LICENSE and TRADEMARK.md: the core is **AGPLv3**; files marked '@license Enterprise' fall under a commercial license requiring an Enterprise subscription for production use; several packages (twenty-ui, twenty-sdk, twenty-client-sdk, create-twenty-app, twenty-shared) are MIT; and the 'Twenty Application Exception' to AGPLv3 section 7 exempts apps that interact only through published HTTP/GraphQL APIs, webhooks, manifests, and SDKs - so an agent platform integrated over Twenty's public APIs does not become AGPL, but **modifying Twenty itself does**. Trademarks are reserved by Twenty.com, PBC ('built on Twenty' attribution is the allowed form). Verdict: **5/5 as software, wrong foundation for our MVP** - forking a 38,000-file CRM to be an agent platform buys us a data model we do not need yet at the cost of a fork we cannot afford; we take its metadata-driven and agent-thread schema ideas (chapter 11) and keep it as a candidate integration target for CRM-adjacent workflows in Stage 5.

3.6 Build-versus-integrate decision

Layer	Best candidate	Decision	Why
Sandbox execution	rakazo adapters / E2B / plain Docker	Integrate (thin wrapper)	Apache-2.0/MIT paths exist; per-run Docker first, E2B when managed scale is cheaper than ops.
Persistent-agent product surface	rakazo	Reference, do not fork	Closest to target product but operator-shaped (self-host), not multi-tenant SaaS; our audit log is the differentiator.
Coding-agent substrate	OpenHands agent-server	Adopt at Stage 5	MIT, clean REST service, ACP multi-agent; embed only when engineering agents ship.

Layer	Best candidate	Decision	Why
Design reference	OpenDots, coworkers	Pattern source	MIT; approval cards, one-store-per-agent, takeover, spend metering.
Business data / CRM	twenty	Integrate via APIs (Stage 5+)	AGPL core + Application Exception: API-side integration is safe; forking is not.
Orchestration core	Custom (LangGraph-inspired)	Build thin	Audit trail, state machine and budget gates are the product; framework lock-in is premature (chapter 11.1).

Table 3.1 - Reuse decision per layer (decision rationale in chapter 11)

Two architectural conflicts deserve explicit warning. First, **do not combine the four agent platforms**: OpenDots, rakazo, coworkers, and OpenHands-as-workforce each define their own agent identity, computer, and runtime model - adopting more than one means reconciling duplicate identity systems (our inference from the code structure). Second, stack gravity: rakazo (Hono + oRPC + Prisma) and Twenty (NestJS + TypeORM + GraphQL) would each drag a different ORM and API style into the codebase; our MVP keeps one stack (FastAPI + SQLAlchemy + Postgres) and consumes external systems only through their network interfaces.

Sources

github.com/CopilotKit/OpenDots (+ README, LICENSE, package.json, docs/SETUP.md, docs/COMPUTERS.md); ungh.cc mirror metadata (accessed 2026-11-04)

github.com/elie222/rakazo (+ README, LICENSE, package.json, packages/adapters, docs/models.md, docs/computer-runtime.md, VISION.md)

github.com/OpenHands/OpenHands (+ docs/architecture.md, LICENSE) and github.com/OpenHands/software-agent-sdk

github.com/fayazara/coworkers (+ README, LICENSE, docs/desk-plan.md)

github.com/twentyhq/twenty (+ LICENSE, TRADEMARK.md, packages/twenty-ui/LICENSE, twenty-agent-skills/README.md)

Product and UI/UX Blueprint

The interface's job is to make genuine agent work legible and trustworthy: what each agent is doing right now, what it found, what it needs from a human, and what was verified. The design stance follows the evidence in chapter 2: Grok Bot proves users accept named agents with jobs; Muse's reception proves they punish opaque memory and permission behavior. Every screen below therefore exposes state, evidence, and control - and none of them decorate. The information architecture has five top-level areas: **Today** (dashboard), **Agents**, **Work** (tasks and runs), **Approvals**, and **Knowledge** (memory, sources, artifacts), with Settings and an admin console behind them.

4.1 Screen inventory and purpose

Screen	Content and primary interactions	Backend dependency
Today / dashboard (M)	Brief cards (delivered/verified), running-task rail with live state chips, approval inbox count, spend this week vs budget. Actions: open run, approve, pause.	GET /dashboard; WS events; GET /approvals?status=pending
Agents list (M)	Agent cards: avatar, name, role, current state, next scheduled run, 7-day success rate. Actions: create, pause, edit charter.	GET /agents; agent.state via WS
Agent profile (M)	Charter (versioned markdown), skills with versions and permissions, memory entries (typed, deletable), run history with outcomes, per-agent settings.	GET /agents/{id}, /agents/{id}/memory, /runs?agent={id}
Create agent (M)	3-step flow: role template or blank; charter editor with preview of system prompt; tools + permission preset. Ends with a one-task test drive.	POST /agents; POST /skills/{id}/test
Marketplace (S3)	Skill and agent-template gallery with versions, permissions summary, author, install count. Actions: inspect diff, install into agent.	GET /marketplace; POST /agents/{id}/skills
Task detail / run view (M)	Plan (subtasks with acceptance criteria), live step stream (tool call, observation, evidence links), cost and duration, verification panel, artifact preview.	GET /runs/{id}; WS /runs/{id}/events; GET /artifacts/{id}
Browser viewer (S2)	Embedded screenshot stream of the sandbox browser; human can take over or annotate; session recording link.	WS /sandboxes/{id}/stream; POST /sandboxes/{id}/takeover
Files / artifacts (M)	Artifact browser (reports, datasets, screenshots) with previews, download, and provenance (which run produced it).	GET /artifacts; object storage signed URLs
Approvals inbox (M)	Queue of pending actions grouped by risk tier: what, why, agent, blast radius, approve/deny/modify with comment; batch approve for low-risk groups.	GET /approvals; POST /approvals/{id}/decision

Screen	Content and primary interactions	Backend dependency
Memory / knowledge (M)	Search across memory entries and ingested sources; entry inspector: origin run, timestamp, confidence, delete/forget.	GET /memory?q=; DELETE /memory/{id}
Settings / integrations (M)	Workspace members + RBAC roles, connected apps (OAuth grants with scopes), model providers, budget caps, notification routing.	CRUD /integrations, /members, /budgets
Admin / audit (M)	Audit log (who/what/when per action), cost ledger per tenant/agent, watchdog events, feature flags.	GET /audit; GET /ledger; GET /watchdog/events

Table 4.1 - Screen inventory (MVP screens marked M; later stages marked by stage number)

4.2 Design system

The system is tokenized and deliberately boring, in the 'Invisible Precision' tradition: quiet surfaces, one accent, typography that stays out of the way. **Layout:** a 12-column grid on a 1440 desktop canvas with 24px gutters; content max-width 1200px; 8px spacing scale (4/8/12/16/24/32/48); left navigation at 240px, collapsible to icons at tablet widths. **Type:** a single sans family (Inter or system equivalent) with a scale of 12/13/15/18/22/28px for caption/body/strong/subtitle/title/display, line-height 1.5-1.6 for body and 1.2 for headings; tabular numerals for all metrics. **Color:** one blue family matching this document's palette (deep #1a4a7a for headers, accent #2d7ab3 for interactive elements, #e4ecf5 surfaces, #142840 text at AA contrast), plus semantic green/red only for verified/failed states; no more than five colors anywhere. **State chips** are the core visual language: every agent and task is rendered as chip + dot, where color encodes state class (running, waiting, done, failed, paused) and the dot pulses only when an event stream is genuinely active.

Accessibility: WCAG 2.1 AA minimum - all text at or above 4.5:1 contrast, focus rings on every interactive element, full keyboard traversal (agents list, run view, approvals are operable without a mouse), prefers-reduced-motion honored by replacing all pulse/progress animations with static badges, and screen-reader announcements on state transitions ('Research agent moved to Verifying'). **Responsive:** the dashboard, approvals, and run views degrade to single column at 768px; the run stream and browser viewer are desktop-first because operators use them on large screens; approvals are mobile-first because they are latency-critical and often handled from a phone - mirroring Grok Bot's phone/desktop parity lesson. **Motion:** 150-250ms ease-out transitions for panel and state changes; nothing loops except the live-activity dot, which is event-bound (chapter 5.3).

4.3 Component inventory

Component	Displays	Interactions	Backend
StateChip	Agent/run state + live dot + duration	Click opens run view	WS run.state events

Component	Displays	Interactions	Backend
TaskCard	Goal, agent, schedule, last outcome, cost	Assign, pause, edit, rerun	Task + Run APIs
StepStream	Ordered tool calls with observation summaries and evidence thumbnails	Expand step, open evidence, copy citation	WS run.events (append-only)
PlanPanel	Subtasks with acceptance criteria and status	Edit plan before approval; add constraints	Run.plan (versioned)
VerifyPanel	Checks run: pass/fail, evidence per check, verifier identity (which model/bot, not the author)	Re-run verification; challenge a check	GET /runs/{id}/verification
ApprovalCard	Action diff, blast radius, agent, expires-at	Approve / deny / modify / escalate	POST /approvals/{id}
MemoryEntry	Typed fact with origin, confidence, age	Edit, delete, pin, forget-all	Memory API
CharterEditor	Versioned markdown with diff view	Edit, save-as-v(n+1), rollback	Agent.charter versions
CitationChip	Claim-linked source: domain, date, archive status	Hover preview; open archived copy	Artifact + archive store
SpendMeter	Budget vs usage by tenant/agent/day	Set caps, hard-stop toggle	Cost ledger
ScheduleEditor	Cron/timezone preview + event triggers	Add trigger source, test-fire	Trigger API
TakeoverBar	Live sandbox browser with hand-back button	Take over, annotate, release	Sandbox WS

Table 4.2 - Core components (purpose, information, interactions, backend)

4.4 Critical journeys, end to end

Journey A - **assign a research task** (the core loop): user picks an agent and types a goal ('Monitor Figma's pricing page and changelog for changes; draft Friday's brief'). UI shows a plan draft within ~20 seconds (understanding → planning states stream in); user edits acceptance criteria; on 'Approve plan' the UI sends POST /tasks {agent, goal, plan, criteria}; backend validates permissions and budget, enqueues a run, and the run view opens; StepStream renders tool calls live; the run transitions Verifying, and VerifyPanel lists each check with evidence; on Delivered, a CitationChip row appears per claim, and the brief lands in the user's connected Slack and email. Every transition is a WS event the user can replay later from the run history - there is no state in the UI that the event log cannot reproduce.

Journey B - **human approval gate**: an agent proposes signing up for a competitor's trial (ask-first tool). The run pauses in Awaiting Approval; an ApprovalCard appears in the inbox and pushes to Slack/mobile; the card shows the exact form fields the agent intends to submit, the email to be used, and a blast-radius

note ('creates an account; no payment'); the human approves, modifies (changes the email), or denies with a comment; the decision event resumes or terminates the run, and the reasoning is written to the audit log. Approvals expire after 24 hours (the coworkers pattern) to prevent zombie holds.

Journey C - **install a skill from the marketplace**: user opens a skill page showing version, what it does, required tools, and required permissions ('browser.read, http.post api.hubspot.com'); 'Install' shows a permission-consent sheet (OAuth-style); on confirm, the skill lands pinned to that agent with a version tag; the agent's next plan references it; uninstall/rollback is one click back to the previous version. Digest-pinning (the coworkers pattern) prevents silent auto-updates.

4.5 Confusing flows found in the wild, redesigned

Bot vs template vs skill (Grok Bot): the marketplace sells 'bots' that are actually recipes of skills + memories + plugins, while installed artifacts are bots - users cannot tell what will change when they install. Redesign: one **Agent** entity with a versioned Charter, and marketplace items are skills or charter-templates with an explicit diff preview at install time. **Memory sprawl (Muse):** memory/dreams and bank folders are opaque to users and became a privacy scandal. Redesign: typed memory entries (fact, preference, procedure, source) each with origin run, confidence, and a visible delete/forget control, plus a workspace-level 'what does this agent remember about me' inspector. **Approval fatigue:** per-action approvals for everything trains users to rubber-stamp (and Grok Bot's review-agent model makes the model the gatekeeper). Redesign: risk-tiered approvals - read-only actions auto-allowed and logged; external side-effects ask once per action-class with remembered scope; payment and credential actions always ask, with batch approve only for the low tier.

Sources

Pattern sources: Grok Bot guides and marketplace payload; mouse.dev Muse runtime export; coworkers docs (desk-plan, skills, spend); Cursor docs (customize, cloud-agent) - all as cited in chapters 2-3, accessed 2026-11-04

Agent Avatars and Genuine Behavior

An agent's identity and its visible behavior are trust infrastructure, not decoration. The identity model: every agent has a name, a role title, an avatar (a simple generated glyph + color from the system palette - no photorealistic humans, which invite parasocial trust the system has not earned), and a **charter**: a versioned markdown document containing its mission, scope, rules, and current skills. This adopts the readable-identity pattern observed in both Muse (SOUL.md / IDENTITY.md / USER.md / MEMORY.md files in the runtime, verified by the mouse.dev teardown) and Grok Bot's three-field bot definition, with one deliberate improvement: charters are versioned with diffs, so 'what changed about this employee' is always answerable - the governance answer to the privacy criticism Muse received.

5.1 The operational state model

States are not UI fictions; each one is a backend fact with a trigger. The table below is the contract between the agent runtime (chapter 6) and the interface: the runtime emits exactly these state events, and the UI renders exactly what the event says. Eighteen states cover the full lifecycle; four of them (queued, understanding, planning, working) are what other products blur into a single spinner, which is precisely what makes their progress bars dishonest.

State	Backend trigger (event emitted)	UI shows	User actions
Idle	Agent has no queued work; schedule empty	Dormant chip, 'next run: Fri 07:00'	Edit charter, assign task
Queued	Task accepted into run queue, no worker lease yet	Position-in-queue chip, "waiting for capacity"	Cancel, reprioritize
Understanding	Run lease acquired; goal-parsing step active	Goal restated + extracted constraints appearing	Amend goal (re-queues)
Planning	Plan generation; skill/tool selection	Plan draft streaming into PlanPanel	Edit plan, approve, reject
Researching	First web/search tool calls begin	Search queries + source cards appearing	Watch, steer via comment
Browsing	Sandbox browser session active	Screenshot stream + URL bar (sandbox truth)	Take over browser
Thinking	Model-only step (no tool) exceeding ~5s	Subtle 'reasoning' chip, elapsed timer, no fake text	None (transient)
Coding	File edits or terminal commands executing	Diff stream / command output	Open in editor
Writing	Long-form generation for a deliverable	Draft preview updating	Inject style note
Analyzing	Data/table transforms running	Progress by batch, row/record counts	None
Collaborating	Message sent to/received from another agent	Thread snippet + counterpart avatar	Read thread

State	Backend trigger (event emitted)	UI shows	User actions
Awaiting approval	Permission gate requires human decision	ApprovalCard; run paused; expires-at countdown	Approve/modify/deny
Verifying	Verification skill executing checks	Checklist appearing with per-check evidence	Challenge a check
Completed (Delivered)	All checks passed; artifact published	Delivered chip; artifact + summary + citation row	Accept, request changes
Failed	Unhandled error or checks failed after retries	Failed chip + error class + last good step	Retry, escalate, edit plan
Recovering	Watchdog or retry policy re-spawned the run	Recovered-from-step chip	Cancel
Paused	User or budget governor suspended the run	Paused chip + who/why	Resume, cancel
Cancelled	User/budget/timeout terminated the run	Cancelled chip + partial artifacts retained	Rerun

Table 5.1 - Agent/run states: trigger, UI presentation, user controls

5.2 How progress, logs, and results are communicated

Three layers, each answerable from the event log. **Live layer:** the StepStream renders each tool call as a compact row - tool name, one-line arguments summary, observation summary, latency, and cost - so a run that is genuinely browsing shows URLs and screenshots, and a run that is stuck shows repeated identical tool calls (the honest signal). **Summary layer:** when a run completes, the UI replaces the stream with a 5-8 line activity summary written from the structured trace (steps taken, sources consulted, checks passed, cost), which is what gets shared into Slack - a deliberate echo of Cursor's practice of logging everything and replaying sessions. **Audit layer:** the full ordered event log (state changes, tool calls, approvals, token counts) is immutable and exported per run; the UI never shows model chain-of-thought - the trace shows *what the agent did and observed*, not its private monologue, which protects the vendor, the user, and the model provider's terms alike.

5.3 Never fake activity - the integrity rules

The Muse/Grok Bot era has already shown users punish performative agents. We adopt five enforceable rules. **(1) Every animation is event-bound:** the pulsing dot renders only while a WebSocket event stream is open and fresh (heartbeat timeout 10s); no stream, no pulse - an idle agent looks idle. **(2) Progress is counted, not estimated:** progress = verified completed steps / planned steps from the run's own state machine; when the plan changes, the bar re-bases visibly. We do not show a smooth fake curve. **(3) Tool truth over theater:** the browser viewer shows actual sandbox screenshots; 'Thinking' shows a timer, never fabricated prose. **(4) Completion is verified completion:** the Delivered state renders only after the verification skill's checks pass - self-reported 'done' shows as 'Agent reports done (unverified)' in a distinct amber chip. **(5) Distinguish the five ways work can be 'not moving':** queued (no capacity), paused (human), awaiting approval (human gate), rate-limited (external), and stuck (no state change in N minutes - the Watchdog flips this automatically). Each has a different chip, because each has a different remedy.

Sources

Identity-file evidence: mouse.dev Muse runtime export (2026-09-22); Grok Bot bot-definition and guides (x.ai, 2026-11-04)

Integrity rules informed by: Cursor self-driving-codebases observability practice (2026-02-05); coworkers loop-guard and takeover patterns (2026-10-31)

The Agent Harness

The harness is everything around the model: context assembly, tools, permissions, memory, state, orchestration, execution, verification, and recovery. Models are rented commodities; the harness is the product. This chapter defines our harness precisely, because every later chapter (skills, multi-agent, monitoring, roadmap) builds on it. The controlling idea, borrowed from the strongest systems we inspected: **the harness, not the prompt, determines reliability** - SWE-agent's authors showed that the agent-computer interface alone moved SWE-bench outcomes by double digits (arXiv:2405.15793), and Cursor's pstack principle 'prove-it-works' operationalizes verification as a harness duty, not a hope.

6.1 The complete lifecycle



Figure 6.1 - The harness loop: ground, act, verify, learn. Every arrow is an event in the run trace.

Figure 6.1 traces the canonical lifecycle. **Goal** arrives from a human, a schedule, or an event trigger, and is written to the task table with acceptance criteria. **Planning** decomposes the goal into subtasks, each with its own criteria and a budget; plans are objects the user can edit before approval. **Context retrieval** assembles what the model needs now: memory search results, linked thread summaries, and skill content - using Inline & Spill discipline (likely-needed facts inline; large content behind typed references; MemGPT and the DeepSeek article converge on this paging pattern, arXiv:2310.08560 and Nov 2026 article). **Skill selection** attaches the matched SKILL.md playbook - steps, tools, and acceptance checks. **Tool selection and**

permission check run before every side-effectful call: the permission engine evaluates the action against the agent's tool grants, workspace policy, and risk tier, pausing the run for Approvals where required. **Execution** happens in a sandboxed runtime (browser, terminal, APIs) with timeouts and output caps; every call and observation is written to the run trace. **Evaluation** after each step asks 'did this move the goal,' enabling correction loops; **verification** runs an independent check suite against the acceptance criteria; and **delivery** publishes the artifact with evidence and emits the result event. A gated **learning** phase (chapter 9) may then propose memory or skill updates.

6.2 What the harness provides, concretely

Capability	Design	Basis / evidence
Tool calling	Typed JSON-schema tools; arguments validated; results normalized to structured observations with truncation rules	ReAct loop (arXiv:2210.03629); provider function-calling APIs
Planning	Plan as data: subtasks, criteria, budgets; editable; versioned per run	Grok Bot engineering-lead bot's board/CI gates (x.ai, Nov 2026)
Short-term memory	Run-scoped scratchpad, rewritten not appended when large (freshness pattern)	Cursor self-driving-codebases (2026-02-05)
Long-term memory	Typed entries in Postgres + pgvector: fact/preference/procedure/source; origin and confidence stored; deletable	Generative Agents reflection (arXiv:2304.03442); Muse lesson (ch. 2.3)
Context engineering	Inline & Spill + Chain of Recall over object IDs (task, thread, session)	DeepSeek article (Nov 2026); MemGPT (arXiv:2310.08560)
Skills	SKILL.md + manifest: version, permissions, tools, tests; registry with rollback	pstack (30+ skills, verified); AAIF Agent Skills standard (2026-10-22)
Browser/API access	Sandboxed Playwright + search APIs + MCP servers; egress allowlist per agent	Ch. 8 stack decision
Files/terminal	Sandbox filesystem per run; artifacts promoted to object storage on delivery	OpenHands runtime; rakazo computers
Sandboxed execution	Per-run container: no shared state, network-restricted, resource caps	Firecracker <125ms start (firecracker docs); E2B per-second pricing
Persistent tasks	Task/run rows + checkpoint after each step; crash resume from checkpoint	LangGraph checkpointing model (docs, 2026)
Async execution	start()/stop() semantics for long tools; results delivered as events	DeepSeek full-duplex design (Nov 2026)
Retries/recovery	Classed errors, bounded retries, Watchdog restarts from checkpoint	Ch. 7 Watchdog; ch. 12 SLOs
Human approval	Risk-tiered gates; remembered scopes; 24h expiry	Ch. 4.5 redesign
Verification	Independent verifier (different model/agent) runs the check suite; evidence archived	'prove-it-works' (pstack); evaluator/optimizer split (ch. 9)

Table 6.1 - Harness capabilities and where they live

6.3 Chatbots, tool-callers, task agents, persistent systems

Class	Definition	Example	Our exposure
Chatbot	Single-turn or multi-turn conversation; no tools or state	Support FAQ bot	None - we do not ship one
Tool-calling model	Model + function calling in one session; no persistence	A search-assisted chat answer	A primitive inside the harness
Task agent	Bounded goal -> plan -> tools -> verified artifact; session-scoped	Our research run	The MVP unit of work
Persistent autonomous system	Long-lived identity; triggers; memory; multi-task over weeks	Our scheduled intelligence agent	Stage 2-4 product

Table 6.2 - A taxonomy the team should use in all docs and sales conversations

6.4 The autonomy ladder

Autonomy is granted per action class, per agent, per workspace - never globally. **Autonomous by default** (logged, no gate): read-only web research, searches, reading public pages, drafting, running tests in the sandbox, writing memory entries. **Ask-first** (approval on first use, remembered scope): creating external accounts, sending outbound email or Slack messages to non-team recipients, installing skills or MCP servers, spending above a per-run cap. **Always ask**: payments, credential entry, deleting or overwriting external data, contacting humans outside an approved list, and any action on a source domain the workspace has flagged. **Human-only** (agents propose, humans dispose): publishing the weekly brief externally, changing another agent's charter, granting new tool scopes, and approving skill-version promotions in the improvement loop. This ladder is the direct answer to the incident record in chapter 2 - shared computers and un-gated side-effects are where the shipped products got hurt.

Sources

SWE-agent ACI: arXiv:2405.15793 (2024-05-06); pstack principles: raw.githubusercontent.com/cursor/plugins (2026-11-04)

DeepSeek Harness super-agent article full text via fxtwitter API (2026-11-01, accessed 2026-11-04); MemGPT: arXiv:2310.08560

AAIF Agent Skills donation: Linux Foundation announcement via Google News index (2026-10-22)

Super-Agent Architecture: Router, Workers, Events

This chapter analyzes the article 'Exploring Super-agent Design: Towards a More Autonomous Agent System' as an architectural proposal. Provenance, verified: the English text is a long-form X article posted 2026-11-01 by the verified @DeepSeekHarness account ('DeepSeek Harness'; 13.4k likes, 2.85M views at fetch time), machine-translated from Chinese, explicitly framed as 'discussing architectural ideas... not an introduction to existing features,' with a companion prototype repo (deepseek-ai/dsh-superagent) that the article itself calls 'only a prototype for validating the technical approach, not an official DeepSeek product.' Authorship credit: Yujia Qiao implemented and wrote, based on DeepSeek Harness team work. The reported experiments use synthetic workloads with **simulated** coding execution and are labeled 'indicative only' by the authors. We treat it accordingly: a serious systems design from a team shipping an agent harness, not a product announcement.

7.1 Mechanisms A-G, verified and evaluated

A. Two-level architecture. A persistent Router - 'a long-lived global routing layer that receives human interaction from every entry point... the system's global sequence point, coordination point, and memory' - never executes tasks (it 'can run without a Bash tool at all'), while isolated Workers each own one task with full tools and 'never interact with humans directly.' Follow-ups route to the owning Worker via Thread subscriptions; Workers can spawn subagents or act as teams 'transparent to the Router layer above it.' Conflicts are handled primarily by co-subscribed Workers seeing the same events ('a chance to spot and resolve the conflict on their own'). Evaluation: **adopt** - this matches what Cursor ships operationally (cloud agents + automations) and fixes the documented failure mode of single-session delegation (below). The Router is explicitly not an 'Agent Team' and a lighter model suffices - consistent with our cost model.

B. Event-driven architecture. Inputs are semi-structured triggers (group chats, email, calendar, review comments, CI results). A deterministic **Fast Path** (code policies: allowlists, per-source modes - 'respond only to explicit requests' / 'treat as candidate events' / 'ignore') drops noise before any LLM call; a semantic **Slow Path** has an LLM judge candidates 'based on... policies written as prompts,' doubling as a natural-language control surface. Threads group consecutive events on the same PR/IM topic/email chain; when the Router creates a Worker for a thread's first event, the Worker subscribes, and follow-ups either pass through directly (Subscribed/Fast) or route through the Router's global-state observation (Unsubscribed/Slow); subscriptions are many-to-many. Evaluation: **adopt in Stage 2 (fast path + schedules) and Stage 4 (semantic path)** - the Fast Path is also our cost-control mechanism, and the article quantifies pass-through savings below.

C. Context architecture. 'Treat the limited context window as a cache': **Inline** likely-needed information; **Spill** large content behind references; a stepped procedure fills context by locality (IDs of related objects, then adjacent content such as surrounding IM messages or neighboring review comments), spilling the overflow 'leaving a pointer to where it continues.' Every object gets a unique ID, and injected objects carry

the IDs of related objects, forming a reference graph - the **Chain of Recall** - so the model can 'follow these references and recall context hop by hop.' Evaluation: **adopt wholesale**; it is MemGPT's virtual-context idea made concrete for a multi-object workspace, and it composes with our typed memory.

D. Full-duplex execution. Replace blocking tool calls with asynchronous start()/stop(): 'start() returns Started immediately... when the tool finishes in the background, the system delivers its final result to the session as a new event message.' DeepSeek Harness has this implemented but not default-enabled (article, verified). Per-channel response styles differ: IM replies are capped ('one simple, effective hard rule we have verified is a cap of 100 words per reply' - though no data is shown), while async channels (GitHub/email) get structured, information-dense output. Evaluation: **adopt**; our run-event architecture is already event-delivered, and the channel-style split matches chapter 4's notification design.

E. Compute, state, environment separation. Two decouplings: (1) stateless agent-loop compute in elastic containers with persistent task state in a database; (2) execution environments stay where they are - cloud sandboxes, or user machines reached through 'a lightweight daemon, which keeps no session state and does nothing but execute commands and return results.' BYOM ('Bring Your Own Machine') lets a personal device be an execution target, including devices that are 'only intermittently online': when it reconnects, 'the cloud agent automatically starts and completes the tasks that need to run on that device.' The article positions this alongside Anthropic's 'Decoupling the brain from the hands' and admits the trust assumption: 'the cloud and the endpoints trust each other,' and a compromised Agent Loop 'could take over' every connected endpoint. Evaluation: **defer BYOM beyond Stage 5** - for our enterprise trust model, cloud sandboxes only until a customer demands private execution, at which point the VPC path (Anthropic-style) is safer than endpoint daemons.

F. Independent Watchdog. 'A persistently online distributed system rarely stays reliable for long if it relies only on its internal logic.' The Watchdog is 'an independent Watchdog Session that runs outside the main system and is driven by scheduled messages,' monitors infrastructure failures and whether each Worker 'is making substantive progress or drifting from the overall goal' (the article's key insight: many anomalies are not crashes but unproductive loops), keeps 'rechecking transient external failures until they are resolved,' and - being a full agent session - supports natural-language post-mortems and live correction ('adding or removing prompts or adjusting trigger rules' while the system runs). Note: the article describes the Watchdog but runs no experiment on it. Evaluation: **adopt early (Stage 2)**, because it is also our cost governor and stuck-run detector; implement deterministic checks first (no state change in N minutes, budget overrun, repeated identical tool calls) with model-judgment only for drift assessment.

G. Speculative execution. 'Acting Ahead': based on environmental signals, the agent starts background work 'before it knows whether the result will be used' - a meeting invitation triggers calendar checks and a draft reply; a policy labeled 'silently reproduce and fix' attempts background repro and fix on issue reports, 'delivers a result only after it has confirmed the issue and found a simple fix. Otherwise, it stays quiet the whole time.' Evaluation: **defer to Stage 5+** with hard budget caps and silent-unless-verified semantics; the article itself lacks a permissions/cancellation treatment for speculative work, which we consider a prerequisite (our inference).

7.2 What the experiments actually showed

Experiment 7.1 (end-to-end ablation, scheduling only): four designs ran on a synthetic stream of ~300 messages, ~20 of them work requests, with coding work **simulated by a time-consuming tool** - so this measures coordination, not task execution quality. Each design ran 5 times. The Super-agent completed work at p50 12.4 / p95 15.7 minutes. A neutral-prompt single session with a subagent tool delegated in 0 of 5 runs (all work interleaved in the main session) and hit p95 29.5 minutes; a delegation-first prompt delegated in 4 of 5 runs but remained slower, failing on runs where the model judged the work 'small.' Token accounting: without pass-through, Router decisions rose ~30% and total cache-read tokens went from 23.2M to 31.8M (+37%) - so pass-through's measured benefit is Router overhead, not task speed. Experiment 7.2 (Router stress): with a real model routing and mock workers, decision rates raised from ~1 to ~40 per minute; batch size grew 1.0 to 2.8 while per-request time rose only 2.5s to 3.9s, so **amortized inference time per decision fell 6.4s to 1.6s** - a single long-lived Router session absorbs bursts by batching merged triggers. Honest caveats stated by the authors: synthetic workloads, simplified environments, single-team scale for the Router, and long-term memory named as the eventual bottleneck (their section 8.1).

-37%

Router cache-read tokens from thread pass-through
(DeepSeek exp. 7.1, Nov 2026)

7.3 Scaling, limits, and what we take

Router bottleneck: the batching result (7.2) plus the authors' own note that events arriving mid-turn are merged into the next LLM call means a single-session Router is sufficient 'at our current team size,' with horizontal sharding as the acknowledged fallback 'until the global memory problem is solved.' Our plan mirrors that: one Router session per workspace (not per deployment), which also bounds the blast radius of Router context mixing between tenants - a multi-tenancy concern the article does not address (our inference). **Agent-to-agent communication:** the design deliberately keeps workers peer-silent and coordinates via shared threads; we adopt this and add our approval/audit hooks at the Router boundary. **Context boundaries:** Chain of Recall gives provenance but also a path for information to hop between threads; per-tenant object-ID namespacing and permission-aware recall are our additions. **Privacy/security:** the article's endpoint-trust gap (E) and the absence of any injection analysis are the two biggest holes; chapter 8's prompt-injection controls and chapter 11's egress-only sandbox network policy are our compensations. **MVP sequencing:** Stage 1 ships single-run task agents (no Router); Stage 2 adds persistent state, triggers with a deterministic Fast Path, and the deterministic Watchdog; Stage 3-4 add thread subscriptions, pass-through, and the semantic Slow Path; Stage 5 adds worker teams and (optionally) speculative execution behind budgets. Nothing in the MVP depends on unproven parts of the article.



Figure 7.1 - The two-level, event-driven architecture as specified by the article, with our annotations.

Sources

Article full text: x.com/i/article/2116829218935857152 via fxtwitter API (2026-11-01; accessed 2026-11-04); prototype: github.com/deepseek-ai/dsh-superagent README

Third-party implementation: github.com/yingpengma/cc-super-agent README (accessed 2026-11-04)

Anthropic 'Decoupling the brain from the hands' as cited within the article

Skills, Browser, and the Multi-Agent Workforce

8.1 The skills registry

A skill is a versioned unit of procedure: a SKILL.md (instructions, steps, tool usage, acceptance checks, stop rules) plus a manifest (name, semver, required tools and permissions, inputs/outputs, test fixture). The registry stores every version immutably, resolves an agent's skill reference to an exact version (digest-pinned - the coworkers pattern that prevents silent drift), supports composition (a skill may invoke another skill as a tool with its own permission context), and rolls back by re-pinning. Discovery is by agent charter (declared skills), by task-time retrieval (the harness searches the registry by goal embedding, Voyager-style - arXiv:2305.16291 - but never silently installs), and by marketplace (Stage 3). Sharing requires the author to declare permissions in the manifest; installation shows the consent sheet from chapter 4's Journey C. Testing is a first-class field: skills ship with fixture runs executed against a sandbox before promotion, which is the practical application of pstack's '/maintain-verification-skill' and its Eval Playbook lesson (chapter 2.5). Version promotion follows the gated loop of chapter 9: proposed diff, independent benchmark, human approval, gradual rollout, rollback.

The initial skill set for the intelligence product (each one exercised in the chapter 14 worked example): **web.search** (SERP grounding), **web.read** (fetch + readability + screenshot), **web.monitor** (diff a source vs last snapshot), **source.archive** (persist immutable evidence copy), **extract.changes** (schema-extract product/pricing/messaging changes), **verify.citations** (re-fetch every claim's source and compare), **brief.compose** (weekly brief structure), **notify.slack/notify.email** (delivery with approval tiers). Ten specialists - Research, Engineering, Browser, Product Designer, UI/UX, Coding, QA, Business Research, Writing, and Project Manager - are expressible as charters over this registry; the PM charter, for instance, is a supervisor that holds thread subscriptions and delegates (chapter 8.3).

8.2 The browser and internet stack

Option	What it is (verified)	Strengths	Limits / risks
Playwright	Browser automation over CDP; the substrate most stacks drive	Mature, free, self-hosted in our sandbox	No stealth; anti-bot friction is ours to manage
browser-use	Python agent-browser library (~120k stars, MIT) + hosted stealth browsers from $0.02/browser-hour; self-reported 90.5% on its own Stealth Bench V2	DOM-extraction loop purpose-built for LLMs	Stealth scores are self-reported; arms race is permanent
Computer-use APIs	Vendor GUI-control APIs (Anthropic doc exists; content geo-blocked to us; OpenAI Operator launched Jan 2025, TechCrunch)	General computer control	Costly, slower; unverified details here - flagged

Option	What it is (verified)	Strengths	Limits / risks
Search APIs	Tavily $0.008/credit, Serper $0.30/1k queries, Brave $4-5/1k (official pricing pages, Nov 2026)	Cheap grounding, structured results	Not a browser; needs fetch+render layer
MCP	Open protocol for tools; spec 2026-07-28 adds stateless HTTP; under Linux Foundation AAIF since Dec 2025; OAuth 2.1 since 2025-06-18	Vendor-neutral integration surface; huge ecosystem	Server quality varies; trust model per-server

Table 8.1 - Browser/search stack options (facts verified Nov 2026 where cited)

Decision: **search API for grounding + Playwright in-sandbox for navigation/extraction + MCP for third-party tools**, with browser-use's extraction loop as a library if its iteration quality proves out in our evals. The real internet workflow is a seven-step skill chain: search, navigate, observe (structured extraction + screenshot), interact (forms within permission tiers), extract, retain sources (immutable archive copy + timestamp), verify (re-fetch and compare). Hard realities we design for, not around: **authenticated sessions** run only in user-consented profiles with per-domain scope (the Grok Bot shared-login lesson is a hard 'no' for us); **CAPTCHA and anti-bot walls** hand the session to the human (takeover flow, chapter 4) rather than attempting evasion - a deliberate integrity stance; **rate limits** are respected via per-domain budgets; **downloads/uploads** are virus-scanned and size-capped; and **prompt injection** is treated as unsolved - because the vendors themselves say it is: Brave writes that 'Indirect Prompt Injection remains a fundamental security challenge' (Jun 2026) and OpenAI told TechCrunch (Dec 2025) that AI browsers 'may always be vulnerable.' Controls: web content enters the model as untrusted quoted data, never as instructions; side-effectful tools require out-of-band confirmation regardless of what the page says; and every external-origin instruction attempt is logged as a security event.

8.3 The specialist workforce and supervision

The MVP workforce is deliberately small: one **Intel Supervisor** (PM charter) and one **Research Worker** per tenant, plus a shared **Verifier** agent that is deliberately a different model from the worker's (the evaluator/optimizer split of chapter 9). Supervision follows the Router pattern of chapter 7: the supervisor owns threads and delegates; workers never message humans; shared artifacts (source snapshots, extracted-change records, draft briefs) live in the workspace object store with provenance; conflicts (two workers editing the same brief) are prevented structurally - one owning worker per thread, others subscribe read-only. Parallelism is expressed as multiple workers per tenant when volume justifies it (Stage 4), each with its own sandbox; the supervisor merges verified contributions into the deliverable. 'Unlimited bots' is therefore framed honestly in all our copy: what scales is **saved agent profiles** (charters), while concurrent executions are bounded by plan quotas (workers, browser sessions, model tokens, per-run budgets) - a positioning identical in spirit to every inspected product's metering (Cursor bills cloud-agent usage; Grok Bot gives bots 'their own usage';rakazo meters per-computer), because something real - compute - is being consumed.

Sources

Stack pricing: tavily.com/pricing, serper.dev, brave.com/search/api (accessed 2026-11-04); browser-use: github.com/browser-use/browser-use, docs.browser-use.com/llms.txt

MCP: modelcontextprotocol.io spec changelogs (2025-06-18, 2026-07-28); AAIF donation (2025-12-09) and Agent Skills addition (2026-10-22) via Linux Foundation/Google News indices

Injection stance: Brave (2026-06-08); TechCrunch quoting OpenAI (2025-12-22); Wiz agentic-browser review (2026-01-16)

Voyager skill library: arXiv:2305.16291 (2023-05-25)

Agentic Engineering and Self-Improvement

9.1 The engineering workflow agents should run

For engineering-agent work (Stage 5), the pipeline we adopt - synthesizing Cursor's documented practice, pstack's playbooks, and OpenHands' substrate - is: **repository inspection** (structure, conventions, AGENTS.md/rules ingestion) to **dependency mapping** to **planning** (small, reviewable change-sets; pstack's 'managing PR sizes' lesson) to **code changes** in a sandboxed clone to **terminal execution** (build, linters) to **tests** (TDD where the codebase supports it; pstack's /tdd) to **debugging** (runtime/trace forensics playbook) to **self-review against a feature map** to **verification** by a project-local verify skill to **pull request** with evidence links to **CI monitoring** with automatic fix-up pushes. Two guardrails matter more than any prompt: strict CI constraints (the Maven course's 'Strict CI Constraints' chapter) so merges cannot bypass checks, and codebase guardrails (lint rules, type strictness) that 'encode lessons in structure' rather than instructions - both direct pstack principles. The human owns every merge, in the words of the Grok Bot engineering-lead template and Cursor's trust-curve framing alike.

9.2 What the research actually supports

The table below is the distilled bibliography. Two classification rules keep it honest. **Weight-level learning** (actual training) changes model parameters: Toolformer's API-call finetuning, Self-Rewarding Language Models' iterative DPO, Meta's Early Experience (SFT-style learning from the agent's own rollouts). **Inference-time learning** changes only context, memory, or skills: Reflexion's verbal self-critique, Self-Refine's polish loops, Voyager's code-skill library, Generative Agents' memory-stream reflection, MemGPT's paging. Harness-level self-modification (DGM rewriting its own code, AlphaEvolve evolving programs) sits between: the model is frozen but the agent improves. Our platform is an inference-time learner by design in the MVP - memory and versioned skills - because that is the cheap, reversible, auditable tier, exactly the tier the evidence favors for production.

Paper (authors)	Date	Result (as stated in abstract)	Lesson for us
ReAct (Yao et al.)	2022-10	+34% absolute success on ALFWorld, +10% WebShop over strong baselines	The trace-act loop is the atomic unit
Toolformer (Schick et al.)	2023-02	Self-supervised API-call finetuning; 6B model competitive with much larger ones	Training vs prompting is a real fork; we prompt

Paper (authors)	Date	Result (as stated in abstract)	Lesson for us
Reflexion (Shinn et al.)	2023-03	91% pass@1 HumanEval, above GPT-4's 80% at the time	Persist failures as text; reuse next attempt
Self-Refine (Madaan et al.)	2023-03	~20% average absolute improvement across 7 tasks	Cheap polish pass; schedule speculatively
Generative Agents (Park et al.)	2023-04	Believable agent behavior; observation+planning+reflection each critical	Reflection makes long-run memory tractable
MemGPT (Packer et al.)	2023-10	OS-style virtual context; docs > window; multi-session memory	Inline & Spill ancestor; typed references
SWE-bench (Jimenez et al.)	2023-10	2,294 issue-PR pairs; best model then 1.96%	A verified harness created an industry
SWE-agent (Yang et al.)	2024-05	12.5% SWE-bench full set via interface design (ACI)	Tool interface quality is a first-class lever
AgentBench (Liu et al.)	2023-08	Large gap commercial vs open models; failures: long-horizon reasoning, instruction following	Eval weight belongs on long-horizon behavior
GAIA (Mialon et al.)	2023-11	Humans 92% vs GPT-4+plugins 15% on 466 questions	'Easy for humans' is the right frontier probe
Self-Rewarding LMs (Yuan et al.)	2024-01	Llama-2 70B beats GPT-4 0613 on AlpacaEval 2.0 after 3 iters	Self-judged rewards need held-out judges
AlphaEvolve (Novikov et al., DeepMind)	2025-06	48-mult 4x4 complex matmul (first Strassen improvement in 56 years); sped up its own training LLM	Evaluator-first loops are the safest self-improvement
Darwin Godel Machine (Zhang et al., Sakana/UBC)	2025-05	SWE-bench 20.0->50.0%, Polyglot 14.2->30.7% by rewriting its own agent code, with sandboxing + human oversight	Archive + empirical validation + oversight is the template
Early Experience (Zhang et al., Meta)	2025-10	Learning from own rollouts without rewards generalizes across 8 environments; foundation for RL	Log events today that can become training data
METR time horizons (Kwa et al.)	2025-03	Frontier agents ~50-min human-time horizon; doubling ~every 7 months	Reliability over time is the KPI
TheAgentCompany (Xu et al.)	2024-12	Best agent completes 30% of tasks in a simulated software company	Long-horizon autonomy is still hard; design for it
Terminal-Bench 2.0 / LHTB (2026)	2026-01/10	Frontier <65% on 89 hard terminal tasks; dense checkpoints reveal 0.221-vs-0.653 reward gap vs binary scoring	Instrument process rewards for the Watchdog
Do Agent Optimizers Compound? (Wang et al.)	2026-07	GEPA gains transfer below baseline on new tasks; only regression-controlled continual loop kept improving (76.4% vs 66.0%)	Regression control is non-negotiable in our loop

Table 9.1 - Key papers (all arXiv links accessed 2026-11-04)

9.3 The safe improvement loop

Synthesizing the survey frameworks (arXiv:2404.14387: experience acquisition, refinement, updating, evaluation; arXiv:2508.07407: optimize inputs, memory, tools, workflow, or weights - each with different risk) with the failure evidence, our loop is: **execution results** (runs, checks, human edits) are logged to the eval store; **evaluation** against the frozen task suite separates signal from noise; a **proposed improvement** is generated as a versioned change (memory entry, skill patch, charter amendment - never raw prompt hacks); an **independent benchmark** runs the change against the held-out suite with a judge that is not the proposing agent; **approval** by a human ships the version with a gradual rollout (5% of runs for one week); and **monitoring with rollback** reverts automatically if success rate or cost regress beyond thresholds. The chapter 9.1 lesson from 'Do Agent Optimizers Compound?' is baked in: gains that do not survive the regression suite are discarded, however impressive they looked in-sample. And the survey lesson stands as strategy: optimize the safest component that yields the gain - for us, memory and skills first, harness code second (DGM-style, sandboxed and archived), weights never (we rent models).

Sources

All arXiv IDs as listed in Table 9.1 (abstracts fetched via arXiv API, 2026-11-04)

pstack README and Maven course chapters (as cited in ch. 2); DGM safety statement in abstract (arXiv:2505.22954)

Real-World Problem Selection

This chapter ranks where an agent workforce genuinely pays, using the demand evidence gathered in November 2026. For each candidate we asked the same eight questions: who has the pain, what they use today and why it falls short, what tools and workflow the agent would run, what the deliverable is, where the human checkpoints sit, how results are independently verified, what business outcome and ROI result, and whether demand is evidenced (funding, pricing pages, case studies) rather than asserted.

10.1 Candidates evaluated

Competitive and market intelligence monitoring. Pain: mid-market B2B teams learn about competitor pricing/feature/messaging changes late; analysts burn 5-10 hours weekly compiling what a machine could watch. Today: Google Alerts (noisy), Visualping at $14-$140/mo (raw page diffs, no synthesis - official pricing, Nov 2026), Crayon/Kompyte (enterprise, sales-gated). Agent workflow: scheduled monitors + extraction + verification + composed weekly brief (chapter 14 is the full design). Verification: citation re-fetch and archived snapshots. Demand: Clay's $7.1B valuation and $167-$446/mo team pricing validate the spend; the verification gap is open. **Lead research and enrichment.** Pain: SDR teams need researched, personalized prospect briefs; Apollo sells $49-$79/user/mo (official pricing) but generic; agencies already pay Clay for this. Agent workflow: firmographic research + verification + CRM writeback with approval. Verification: field-level source citations. Strong demand, more crowded. **Software development.** The most proven agent value (SWE-bench 74-79% Verified by late 2025) but also the most contested - Cursor, Copilot, Claude Code, and OpenHands-ecosystem tools own the workflow; a small team enters only via vertical niches. **QA automation.** Funded wave (QA Wolf $36M, Checkly $20M, Momentic $15M - TechCrunch 2024-2025) attacking flaky-selector maintenance; real pain, but heavy browser-infrastructure competition. **Reporting and data analysis.** Recurring, verifiable, but buyers expect BI incumbents to absorb it. **Customer operations.** Sierra ($950M at $15B) and Decagon ($250M at $4.5B) prove enormous value - and enormous capital requirements; not our lane. **Finance/accounting ops.** Basis's $100M at $1.15B (Feb 2026, Reuters/SiliconANGLE) validates vertical finance agents; domain depth required. **Generic workflow automation.** The Gartner 40%-cancellation zone; avoid.

Opportunity	Urgency	Willingness to pay	Feasibility	Measurability	Verdict
Competitive/market intel briefs	High	High (Clay anchor)	High	High (cited claims)	1 - build this
Lead research briefs	High	High	High	Medium	2 - natural add-on
QA automation	Medium	High	Medium	High	Later, infra-heavy
Engineering agents	Medium	High	Medium	High	Stage 5 vertical
Finance ops agents	High	Very high	Low (domain)	Medium	Not yet
Customer ops	High	Very high	Low (capital)	High	Avoid
Generic automation	Low	Low	High	Low	Avoid

Table 10.1 - Opportunity ranking (1 = best; composite of urgency, willingness to pay, feasibility for a 3-6 person team, measurable value)

10.2 The recommended wedge and the industry backdrop

Recommendation: **the competitive-intelligence brief agent**, with lead-research briefs as the adjacent expansion. It is the highest composite score because the deliverable is a document (not a risky action), the workflow is read-heavy (low blast radius - ideal for a young trust brand), verification is natural (citations and archives), the cadence (weekly) makes reliability visible and improvable, and the incumbent set leaves the 'analyst-grade middle' empty: Visualping is too shallow, Crayon/Kompyte too enterprise, Clay powerful but general and action-metered. Industry backdrop, honestly split: **real** - coding-agent capability gains (Stanford AI Index 2025: benchmark scores up tens of points in a year; agents beating humans on time-limited programming tasks), interoperability consolidation (MCP under the Linux Foundation; OpenAI donating AGENTS.md; Agent Skills standard), and infrastructure maturity (Modal scaling to a million concurrent sandboxes; E2B per-second billing). **hype** - generic platform bets (the Gartner cancellation projection), self-reported stealth/agent benchmarks, and 'autonomous browsing' claims while prompt injection remains fundamentally unsolved. **Underserved** - verification and audit-grade execution, long-horizon reliability tooling, and vertical agents whose every action is checkable: exactly our wedge.

Sources

Pricing/funding: clay.com/pricing; visualping.io/pricing; apollo.io/pricing; SiliconANGLE/Reuters (Clay 2026-09, Basis 2026-02, Sierra 2026-05, Decagon 2026-01); TechCrunch (QA Wolf 2024, Checkly 2024, Momentic 2025) - all accessed 2026-11-04

Stanford AI Index 2025 (hai.stanford.edu); Gartner projection via Forbes (2026-07-07); swebench.com snapshot (2026-11-04)

Technical Architecture

The architecture optimizes for one constraint above all: a 3-6 person team must be able to debug the whole system in its head. That rules out microservices, rules in a monolith plus workers, and makes the audit log - not the framework - the spine. All technology choices below name the rejected alternative, because build-versus-integrate is a sequence of explicit bets.

11.1 Stack decisions with rejected alternatives

Layer	Choice	Why	Rejected / revisit trigger
Frontend	Next.js + TypeScript + Tailwind + shadcn/ui; TanStack Query + SSE/WS	Hiring pool, speed; SSE covers 90% of streaming; WS only for browser takeover	Reflex-style realtime first; revisit none
Backend	Python 3.12 + FastAPI monolith; SQLAlchemy; Pydantic	One language for API, workers, and model glue; async-native	Node/Hono (rakazo's stack) - fine, but our model tooling is Python-first; NestJS (Twenty) too heavy

Layer	Choice	Why	Rejected / revisit trigger
Orchestration	Custom thin state machine (explicit states, transitions in Postgres) + Redis Streams queues	Auditability and budget gates are the product; state machine is ~1-2k LOC	LangGraph (MIT, proven at Klarna-scale per langchain.com) - adopt if HITL graph logic outgrows ours; AutoGen (archived by Microsoft, README notice) and CrewAI (role-play shaped) rejected on control grounds; OpenAI Agents SDK strong but provider-coupled habits
Database	Managed PostgreSQL 16 + pgvector; Redis; S3-compatible object storage	One durable store; vector search without extra infra	Dedicated vector DB (Pinecone) deferred until >10M vectors
Sandboxing	Docker containers per run (gVisor if available); E2B or Firecracker at scale	Per-run isolation, egress allowlist, <125ms start (Firecracker docs)	Shared persistent computers (Grok Bot model) rejected on the incident record
Model layer	Provider gateway with 2+ vendors, per-tenant keys, spend caps; verifer on a different vendor	Cost routing + fallback; evaluator independence	Single-vendor lock-in rejected explicitly
Observability	OpenTelemetry (GenAI semconv) + self-hosted Langfuse + Grafana	GenAI span conventions now have a dedicated OTel repo; Langfuse is MIT-core, self-hostable, $29/mo cloud tier	LangSmith (fine, SaaS-first); Arize Phoenix (ELv2 license change noted)
Eval/benchmarks	Frozen internal 50-task suite + citation-re-fetch verifier + human spot-checks	Contamination-proof (SWE-bench lesson)	Public leaderboard chasing rejected

Table 11.1 - MVP stack (all licenses verified Nov 2026)

11.2 System architecture

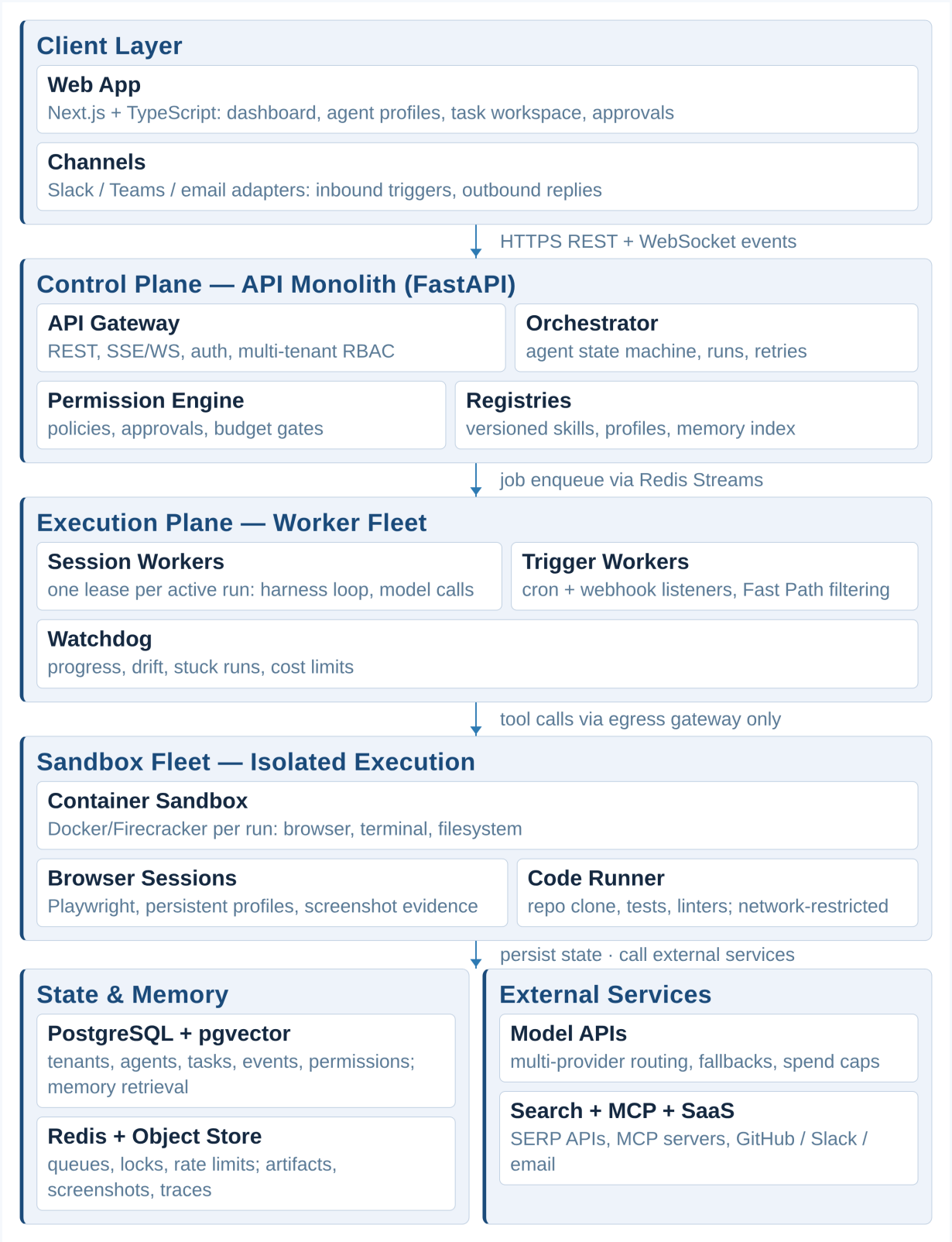


Figure 11.1 - MVP system architecture: control plane, execution plane, sandbox fleet, state, and externals.

Request flow: clients talk only to the API service. Mutations land in Postgres; anything the agent system must react to is published to Redis Streams. Run workers lease a run (SETNX lease key with TTL), execute the harness loop of chapter 6, and stream events to both the run trace (Postgres, append-only) and the WS fan-out. Trigger workers own schedules (a Postgres-backed cron table, polled) and inbound webhooks, applying the Fast Path filter before enqueueing. Sandboxes are created per run, addressed by the worker over a private network, and destroyed on run end; only the egress gateway can reach model/search/MCP endpoints. The Watchdog runs as its own container on a 60-second loop, reading only the queue and run tables - it can restart, kill, escalate, and freeze budgets, and it writes to the same audit log as everything else.

11.3 Data model

Table	Key fields	Notes
tenant / user / membership	plan, rbac_role	Workspaces are tenants; roles: owner/admin/member/viewer
agent	charter_v, status, schedule, tool_grants, budget_caps	charter_v points to immutable charter version
charter_version	markdown, diff_from, author	Full history; rollback = repoint
skill / skill_version	name, semver, manifest, digest, test_report	Immutable versions; digest-pinned installs
task	goal, agent_id, criteria jsonb, trigger_source	criteria drive verification
run	task_id, state, plan jsonb, checkpoint jsonb, cost, started/ended	state machine owner; idempotency_key per step
run_event	run_id, seq, type, payload jsonb	Append-only trace; powers UI replay
tool_call	run_id, tool, args, result_ref, status, latency, cost	Result blobs live in object storage
approval	run_id, action_ref, risk_tier, status, decided_by, expires_at	24h expiry; decision audited
memory_item	agent_id, kind, content, embedding, origin_run, confidence	kinds: fact/preference/procedure/source
artifact	run_id, kind, storage_key, sha256	Reports, datasets, screenshots
source_snapshot	url, fetched_at, storage_key, sha256, content_hash	Immutable evidence for citations
thread / subscription	external_ref, kind, agent_id	Super-agent thread model (ch. 7)
trigger	kind (cron/webhook), config, fast_path_policy	Deterministic filter rules stored as data
cost_ledger	tenant_id, agent_id, run_id, model, tokens, usd	Reconciled with provider invoices
audit_log	actor, action, object, before/after, ip	Who did what, including humans

Table 11.2 - Core schema (simplified; every table carries tenant_id, created_at, updated_at)

11.4 API and event contracts

Endpoint	Purpose	Emits / returns
POST /tasks	Create task (goal, agent, criteria, schedule?)	task; event task.created
POST /tasks/{id}/plan:approve	Approve/edit plan (HITL gate)	event run.planned -> run.started
GET /runs/{id}; GET /runs/{id}/events	Run state; replayable ordered events	SSE stream /runs/{id}/events?live=1
POST /runs/{id}/control	pause / resume / cancel / steer (comment)	event run.control; audited
GET/POST /approvals; POST /approvals/{id}/decision	Approval inbox and decisions	event approval.decided; run resumes
CRUD /agents; POST /agents/{id}/charter	Agent lifecycle + charter versions	event agent.updated
POST /agents/{id}/skills	Digest-pinned skill install	event skill.installed (consent logged)
GET /memory?q=; DELETE /memory/{id}	Memory search / forget	audit entry on delete
POST /triggers; POST /triggers/{id}/test	Schedule/webhook triggers + dry-run	event trigger.fired
GET /ledger?tenant=&agent;=	Spend reporting	reconciles cost_ledger

Table 11.3 - Primary API surface (REST, JWT or session auth, tenant-scoped)

Event architecture: Redis Streams topics **tasks** (new work), **runs.events** (per-run fan-out channel, mirrored to Postgres for replay), **approvals**, **watchdog.commands**, and **audit**. Consumer groups give at-least-once delivery; every handler is idempotent on (run_id, seq). WS clients subscribe to run channels after an authorization check against tenancy. Events are the single source of UI truth (chapter 5's rule): the dashboard is a projection of events, and the same projection powers the customer-facing weekly digest.

11.5 Repository structure and deployment

```
docs/
```

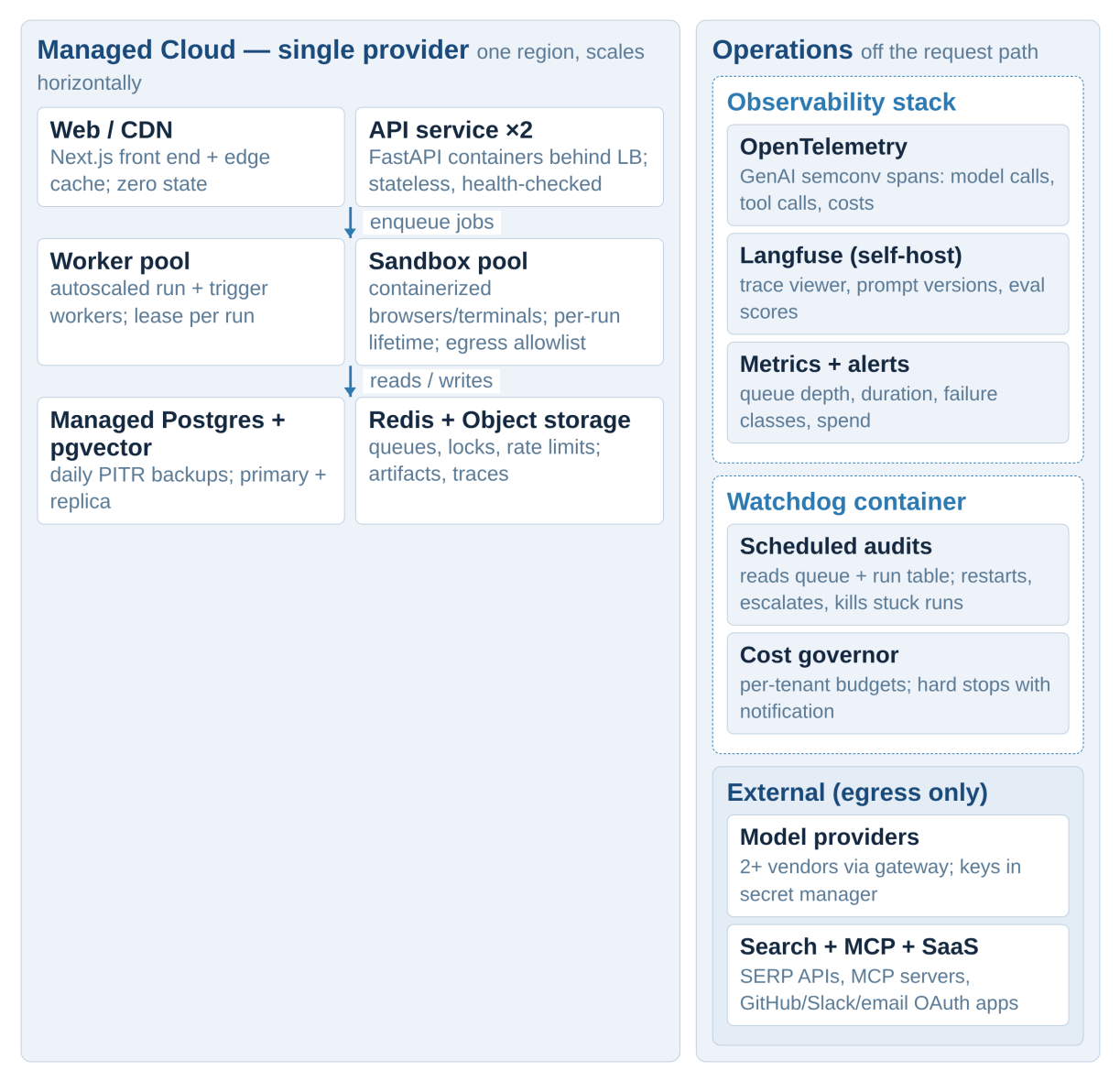


Figure 11.2 - Production deployment: single cloud, stateless services, observability and Watchdog off the request path.

11.6 Security and reliability hard rules

Multi-tenancy and RBAC: every row carries tenant_id with Postgres row-level security as a second net; roles are owner/admin/member/viewer; agent tokens are derived, scoped credentials, never user passwords (the Instinct credential-store model is rejected). **Sandboxing:** per-run containers, no shared computers; network egress only via an allowlist proxy to model/search/MCP endpoints; filesystem and process caps; secrets injected at run start from a secret manager, never stored in model context. **Prompt injection:** web content is quoted untrusted data; side-effectful tools require out-of-band confirmation; injection attempts are security events (chapter 8.2). **Audit:** append-only audit_log for human and agent actions; run traces immutable; retention per tenant policy. **Budgets and idempotency:** per-tenant/agent/run spend caps enforced in the gateway; every mutating endpoint takes an idempotency key; workers hold leases with TTLs so crashes cannot double-run. **Timeouts and retries:** every tool call has a timeout and output cap; error classes (transient/permission/capability/verification) drive bounded retries; the Watchdog escalates repeated failures to humans. **Backups and rollback:** managed Postgres PITR; artifacts and archives in versioned object storage; deploys are immutable images with one-command rollback; skill versions digest-pinned so rollbacks are exact. These are MVP-stage requirements, not aspirations - each maps to a specific incident class documented in chapters 2 and 8.

Sources

Framework facts: microsoft/autogen README archive notice; langchain.com/customers (Klarna); docs.langchain.com persistence (2026); openai.github.io/openai-agents-python (all 2026-11-04)

Infra facts: firecracker docs (<125ms start); e2b.dev/pricing; langfuse.com/pricing + LICENSE; OTel GenAI semconv repo (2026-11-04)

Monitoring, Reliability, and Validation

The monitoring design starts from a definition most teams skip: **what counts as a successful agent task**. Ours: the run reached Delivered, every acceptance check passed under an independent verifier, and the human owner did not reject the artifact within its review window. Everything below serves that number. Telemetry follows OpenTelemetry's GenAI semantic conventions (spans for model calls, tool calls, and MCP operations - the conventions now live in their own OTel repo with MCP attributes), exported to a self-hosted Langfuse for trace inspection and to Prometheus/Grafana for SLO dashboards. Because conventions are still maturing ('Development' status in parts), we version our span schema internally so upgrades are deliberate.

12.1 What we measure

Metric	Definition	Target
Task success rate	successful / total finished runs, per agent and per task type	>= 85% on frozen suite
Correctness	claims verified by citation re-fetch / total claims	>= 95%

Metric	Definition	Target
Verification pass rate	runs passing verify skill on first attempt	>= 75%
Reliability (schedule adherence)	scheduled runs delivered in window / scheduled	>= 98%
Recovery rate	failed runs auto-recovered to success / failures	>= 60%
Time to completion	p50/p95 per task type	p95 < 45 min (research brief)
Cost per successful task	(model + sandbox + search USD) / successes	<= $3.00 (brief)
Human intervention	approvals + edits + escalations per run	<= 1.0 avg
User satisfaction	in-product rating + brief acceptance without edits	>= 70% accepted unedited
Business outcome	customer-reported decisions/weeks citing the brief; retention	>= 60% week-4 tenant retention

Table 12.1 - Metric set (targets are Stage-3 exit criteria)

The **monitoring dashboard** has four panes: live fleet (runs by state, queue depth, sandbox utilization, Watchdog events rolling), reliability SLOs (success rate, schedule adherence, recovery rate against targets with burn alerts), economics (cost per successful task by agent and tenant; spend vs budget with hard-stop markers), and quality (correctness sampling results, human-edit rate, verification failures by check type). Every pane is a projection of the same run_event/cost_ledger tables the product uses - one truth, two views.

12.2 The Watchdog as a reliability component

The Watchdog (chapter 7.1-F) is implemented in two tiers deliberately: **deterministic checks every 60 seconds** - runs with no state change in 15 minutes (stuck), repeated identical tool calls (loop), lease expiry, budget overrun, queue aging beyond SLO, sandbox OOM/timeout patterns - which fire automatic actions (checkpoint restart, kill + retry with backoff, freeze spend, escalate) and write to the audit log; and a **model-judged drift check every 30 minutes** on long-running runs, where a cheap model reads the plan, the last 20 events, and the goal, and answers 'is this run making substantive progress toward the stated criteria' (the DeepSeek article's progress-vs-liveness distinction). The Watchdog is deliberately outside the request path and cannot be invoked by agents; its own actions are, in turn, audited - the article's 'independent Control Loop' requirement, with our multi-tenancy and audit constraints added.

12.3 Evaluation that cannot grade its own homework

Three layers, per the chapter 9 evidence. **Frozen benchmark suite:** 50 research tasks with known-good outputs and checkable criteria, versioned like code, run on every harness and skill change; contamination is controlled by never training on it and rotating 20% quarterly. **Independent verifiers:** the verifier agent runs on a different model family from the worker (the Self-Rewarding lesson: self-judged rewards drift), and the strongest check - **citation re-fetch** - is mechanical, not model-judged: every claim's source is re-fetched, hashed against the archived snapshot, and the passage re-extracted by rules. **Human spot checks:** a weekly 10-claim sample per tenant reviewed by our team against the customer's own judgment, feeding the correctness metric. LLM-as-judge is used only where mechanical checks are impossible, and always alongside agreement monitoring against human labels, because judge bias is documented (MT-Bench literature; position and self-preference effects). **Controlled pilots:** new customers run 4 weeks in shadow mode (agent brief alongside their existing process, double-blind reviewed by their team) before the old process is retired; the delta is the sales proof. We do not - and market-facing claims never will - let the authoring agent be the sole judge of its own success.

Sources

OTel GenAI semconv: github.com/open-telemetry/semantic-conventions-genai; opentelemetry.io/docs/specs/semconv/gen-ai (2026-11-04)

Langfuse pricing/license: langfuse.com/pricing, LICENSE (2026-11-04); MT-Bench judge-bias literature: arXiv:2306.05685 (background, not re-verified this session)

Risks, Costs, and the Phased Roadmap

The roadmap is sequenced so that every stage ships something a paying customer can touch, every stage's components are testable before the next builds on them, and the expensive generic machinery (multi-agent concurrency, semantic routing, marketplaces) arrives only after the reliability core has earned it. Cost estimates assume managed-cloud list prices (heroku/railway/fly-class PaaS or a single cloud account), per-second sandbox billing (E2B-class), and published model pricing; they are order-of-magnitude planning numbers, not quotes.

13.1 Stages

Stage	Build	Cost (infra + models, monthly)	Exit criteria
0. Validate (4-6 wks)	Concierge service: run the weekly brief manually with model APIs for 3 design partners; instrument every hour of labor and every claim verified	< $200 (no product infra)	3 partners renew at $500/mo; >= 80% brief acceptance; documented time saved

Stage	Build	Cost (infra + models, monthly)	Exit criteria
1. Vertical slice (6-8 wks)	Create agent -> assign skill (web.read) -> real tool run -> live status -> verify -> deliver artifact; Next.js + FastAPI + Docker sandbox + Postgres/Redis	$300-700 infra + $100-400 models	Ch. 1.6 pilot gate: 90% on-time briefs, 100% claims re-fetchable, 1 partner migrated to product
2. Persistence & triggers (6-8 wks)	Schedules + webhook triggers with Fast Path; run checkpoints/resume; permission engine + approvals; deterministic Watchdog; OTel traces	$500-1,200 + $300-800	Scheduled fleet runs unattended 4 wks; recovery rate >= 60%; p95 queue wait < 5 min
3. Skills, browser, memory (8-10 wks)	Skills registry with versions/tests; full research workflow (search->verify chain); memory (typed, pgvector); self-serve onboarding + billing	$800-1,800 + $800-2,000	Ch. 1.6 product gate: success >= 85%, cost/brief <= $3, week-4 retention >= 60%
4. Router & multi-agent (8-10 wks)	Thread subscriptions + pass-through; semantic Slow Path; supervisor + parallel workers; marketplace v0 (private skills)	$1.5-3k + $2-5k	Ch. 1.6 business gate: $15k MRR; two threads/tenant concurrently without conflicts; Router p95 decision < 4s at 40 events/min (exp. 7.2 envelope)
5. Engineering agents (10-12 wks)	OpenHands agent-server embed or equivalent; repo workflow of ch. 9.1; CRM integration via Twenty APIs; gated skill-improvement loop live	$2-4k + $3-6k	First engineering-agent pilot PR merged via CI with zero human commits; improvement loop regression-safe (ch. 9.3)
6. Production hardening (ongoing)	Multi-region read replica; SOC 2 track; per-tenant sandbox scheduling; eval suite expansion; marketplace public + revenue share	$4-8k + usage	99.5% API SLO; audit export per customer; security review passed

Table 13.1 - Staged plan (duration assumes 3-6 engineers; acceptance criteria are exit gates)

13.2 What we deliberately do not build (yet)

The honest anti-roadmap, with the trigger that would reverse each: **no Kubernetes** until sandbox churn exceeds two instances' capacity (Stage 4-5); **no microservices** - the monolith splits only when team size forces Conway's law (Stage 6); **no BYOM endpoint daemons** (ch. 7-E) until a design partner demands private execution, and then VPC-first; **no speculative execution** (ch. 7-G) until budgets, cancellation, and silent-until-verified semantics exist; **no bot-to-bot marketplace economy** until the audit trail is a selling point rather than a liability; **no fine-tuning or weight training** - the loop of chapter 9.3 is inference-time only, revisited if a partner's failure distribution justifies it; **no vertical expansion** (engineering, CRM, finance agents) before the intelligence wedge hits its Stage-3 gates. Each of these is a real capability that the inspected products ship; each is also where small teams die. Sequencing is the strategy.

13.3 Platform-level pros, cons, and risks

Risk	P x I	Evidence	Mitigation
Frontier vendor ships our wedge free (Grok Bot use-cases cover 'monitor competitors')	High x High	x.ai/bot/use-cases (56 use cases, Nov 2026)	Verification depth + audit export + vertical workflow polish; sell trust, not capability
Model/sandbox costs spike or pricing models change	Med x High	Per-second billing era (E2B/Modal)	Cost ledger + per-run budgets; 2+ model vendors; cache-heavy Fast Path
Prompt-injection incident via monitored web content	Med x High	Brave/OpenAI statements (ch. 8.2)	Untrusted-content discipline, out-of-band confirmations, egress allowlist, injection-event alerts
Reliability ceiling below sales promises	Med x High	TheAgentCompany 30% long-horizon (arXiv:2412.14161)	Shadow-mode pilots; publish our own metrics; SLA only after Stage-3 numbers
Open-source commoditization (rakazo-class) erodes differentiation	Med x Med	rakazo v0.4.0 velocity (Nov 2026)	Differentiate on hosted reliability + verification service, not runtime features
Contaminated/rotting public benchmarks mislead our evals	Med x Med	SWE-bench Verified retirement (Apr 2026)	Frozen private suite + mechanical checks (ch. 12.3)
Two-person-bus-factor on harness core	High x Med	Team-size assumption	Docs-as-code, run-replay tooling from day 1, pair on state machine changes

Table 13.2 - Key risks and mitigations (probability x impact, honest)

The overall con: this platform is **not defensible by architecture alone** - everything in chapters 6-11 is buildable by a funded competitor in months, and the open-source floor is rising. The defensible assets are accumulated: verified-source archives per customer, the failure corpus that tunes the harness, the frozen eval suite, and the trust brand that comes from publishing honest metrics. The pros: the wedge has demonstrable willingness to pay, the required capital is small because we ride commodity sandboxes and models, and the architectural core (audit-first, verification-first) is aligned with the one thing every incident study says the market will pay for.

Sources

Pricing bases: e2b.dev/pricing, modal.com/pricing, langfuse.com/pricing, tavily.com/pricing (2026-11-04)

Risk evidence: as cited in chapters 2, 8, 9, 12

Worked Example: The Competitive-Intelligence Agent

This chapter walks one real business problem end to end - the Stage-0/1 deliverable - with the verification protocol that makes it provable. Customer profile (composite of the design-partner target): a 120-person B2B SaaS company (call them Acme Analytics) selling to RevOps teams; five tracked competitors; a product-marketing lead who spends Fridays compiling intelligence; current tooling: Google Alerts + a Visualping account at $35/month + a spreadsheet. The deliverable: a Monday 08:00 weekly brief in Slack and email, with every claim carrying a citation that opens an archived snapshot.

14.1 Configuration

One agent, **Scout**, charter v3: mission ('produce the Monday competitive brief'), scope (five competitor domains + four comparison pages + two review aggregators), tools (web.search, web.read, web.monitor, source.archive, extract.changes, verify.citations, brief.compose, notify.slack, notify.email), autonomy tier (read-only + ask-first for any outbound notify beyond the owner), budget ($5/run, $25/week), and stop rules (never contact anyone; never log into any site; hand CAPTCHAs to the human). Six **trigger** sources: cron (Mon 06:30 UTC) plus on-demand 'check now' commands. The **thread** model: each weekly run is a thread; mid-week customer comments ('watch their new pricing page') attach to next week's thread via the subscription (chapter 7).

The run, step by step (typical Monday): 06:30 trigger fires the Fast Path (cron = always work). Scout plans: 5 monitors + 2 searches + extract + verify + compose (~18 steps, est. $1.80). web.monitor diffs each competitor's pricing/changelog page against last week's source_snapshot; three changed. web.read fetches each change with full-page screenshot; source.archive stores immutable copies with SHA-256 and timestamps. extract.changes produces structured records (plan names, prices, copy changes) with per-record source references. verify.citations re-extracts each claim from the archived snapshot by rule (not by the authoring model) and marks 21/22 verified; one claim (a G2 review count) fails the re-fetch and is downgraded to 'unverified - source moved' and excluded. brief.compose writes the 600-word brief: 3 material changes, 2 minor, what-it-means notes, and an 'evidence' section linking every claim. notify.slack posts to #competitive with the summary; the full brief with citations lands in email. Total: 41 minutes, $1.94, 22 claims, 21 verified, 0 approvals needed (read-only week), audit trail complete.

$1.94

typical weekly run cost at Stage-1 pricing assumptions (41 min, 22 claims, 21 verified)

14.2 Independent verification protocol

The brief must not grade itself. Four independent checks run outside Scout. **(1) Mechanical citation re-fetch** (rules-based, separate service): every claim's archived snapshot is re-fetched live; hash and passage must match, else the claim is flagged. **(2) Cross-model review:** a second agent on a different model family reads claim + snapshot and answers 'does this source state this claim?' - disagreement demotes the claim. **(3) Human spot sample:** Acme's lead reviews a system-selected 5-claim sample weekly with one click (agree/disagree); disagreement rates feed Scout's correctness metric and our SLA. **(4) The customer's own outcome test:** at week 4 and week 8, Acme answers whether the brief changed any decision (pricing response, sales-call preparation); we record it as the business-outcome metric from chapter 12.1. Failure handling is rehearsed: if a competitor site blocks the sandbox (anti-bot), Scout marks the source 'blocked - needs human session' and the takeover flow from chapter 4 lets Acme's lead authenticate once into a scoped profile; if the run exceeds budget mid-way, the cost governor stops it and delivers a partial brief with explicit gaps rather than fabricated completeness.

What 'works' means, and how we know: Stage 0 exit (chapter 13.1) required three partners renewing at $500/month with at least 80% brief acceptance; for Acme-class customers the Stage-3 targets are: 98% schedule adherence across 8 weeks, correctness at or above 95% on the human sample, cost per successful brief at or below $3.00, and at least one documented decision per month citing the brief. Those numbers are measurable from the audit trail and the ledger - the same tables the product runs on - which is the point: **the proof of work is the system's own event log, verified by something other than the agent that produced it.**

Sources

Verification design informed by: pstack 'prove-it-works'; SWE-bench Verified contamination lesson (ch. 9); evaluator/optimizer separation evidence (arXiv:2607.14004)

Final Verdict and the First Build

The strongest product strategy: sell verified outcomes, not agents. Enter as the audited competitive-intelligence department for B2B SaaS teams - 'every claim cited, every source archived, every week on time' - priced between Visualping and Crayon, expand to lead-research briefs, and only then widen into the general AI-workforce platform the market will eventually demand. The strategy exploits the one structural weakness shared by all inspected incumbents: they scale capability faster than trust, and trust is what mid-market buyers actually purchase - capability is table stakes.

The recommended architecture: the audit-first monolith of chapter 11 - FastAPI + Postgres/Redis + per-run Docker sandboxes, a thin explicit state machine, a two-tier Watchdog, OTel/Langfuse observability, mechanical citation verification - with the super-agent's proven ideas threaded in staged order: Fast Path triggers and checkpoints (Stage 2), thread subscriptions and pass-through (Stage 4), semantic routing and worker teams (Stage 4-5). Nothing in the MVP depends on the article's unproven parts (BYOM, speculative execution, agent teams), and every staged claim maps to a measured result in chapters 7-9.

The first working version (3 weeks, 2 engineers + 1 part-time designer): Stage 0's concierge service hardened into the Stage-1 slice - one tenant, one agent (Scout), three skills (web.monitor, source.archive, brief.compose), the verify.citations checker as a separate service, live run view with the state chips of chapter 5, approvals for the single ask-first action (notify), and the cost ledger. It runs the chapter 14 workflow for three design partners at $500/month. It deliberately lacks: marketplace, memory UI, multi-agent, triggers beyond cron, and any social features. Its purpose is to convert the chapter 1.6 pilot gate into evidence.

The most important risks, ranked: (1) a frontier vendor bundles a good-enough version of this wedge - mitigated by verification depth, audit export, and the vertical workflow polish they will not prioritize; (2) reliability below promise - mitigated by shadow-mode pilots and publishing our own metrics; (3) a prompt-injection incident - mitigated by the chapter 8 controls and blast-radius limits (read-mostly workflow); (4) cost drift - mitigated by per-run budgets and the Fast Path; (5) team bus factor - mitigated by run-replay tooling and docs-as-code from day one.

The independent tests and real-world evidence needed to prove it works: the frozen 50-task suite (regression gate for every change); the mechanical citation re-fetcher (correctness at or above 95%); weekly human spot samples per tenant (correctness against customer judgment); 4-week shadow-mode pilots against each customer's incumbent process (the delta is the sales proof); schedule-adherence and cost-per-successful-brief dashboards (98% / $3.00 targets); and two referenceable customers willing to state, on the record, that the brief changed a decision. When those exist, the claim in this report's title will have earned its verb: an AI workforce that genuinely performs, verifies, and improves useful work.

References

xAI (2026). Introducing Grok Bot, Aug 11. x.ai/news/introducing-grok-bot; Grok Bot guides, use cases, marketplace incl. /marketplace/engineering (accessed 2026-11-04).

Cursor / Anysphere (2026). Docs: Customize, Cloud Agents, Automations, Bugbot, Grok Bot; Blog: Joining SpaceX (Aug 14), Towards self-driving codebases (Feb 5). cursor.com (accessed 2026-11-04).

Tan, L. (2026). pstack: engineering skills and playbooks. github.com/cursor/plugins (MIT; accessed 2026-11-04).

Maven (2026). How Cursor Turned AI Agents Into Better Engineers - free lesson, Lauren Tan. maven.com/p/e23d9c (accessed 2026-11-04).

Instinct (2026). Homepage, Privacy Policy (rev. Aug 26), Terms. instinct.com (accessed 2026-11-04).

Meta (2026). Muse personal AI agent - indexed official pages; HN item 49615537 (Sep 8). ai.meta.com/muse via search index; muse.ai (403 to fetch).

James, P. (2026). I asked Meta's Muse for its filesystem and it sent me 6.8 GB. mouse.dev/blog/muse-runtime-export (Sep 22; accessed 2026-11-04).

DeepSeek Harness (2026). Exploring Super-agent Design: Towards a More Autonomous Agent System. X long-form article, Nov 1; full text via fxtwitter API; prototype: github.com/deepseek-ai/dsh-superagent (accessed 2026-11-04).

CopilotKit (2026). OpenDots - always-on AI coworkers template (Alpha, MIT). github.com/CopilotKit/OpenDots (accessed 2026-11-04).

Steinbock, E. (2026). rakazo - open-source Grok Bot alternative (Apache-2.0), v0.4.0 Nov 1. github.com/elie222/rakazo (accessed 2026-11-04).

OpenHands (2026). Agent Canvas v1.34 + software-agent-sdk (MIT; SWE-bench badge 77.6 as claimed). github.com/OpenHands/OpenHands (accessed 2026-11-04).

Ahmed, F. (2026). coworkers - AI teammates on Cloudflare (MIT, experimental). github.com/fayazara/coworkers (accessed 2026-11-04).

Twenty (2026). Open-source CRM v2.54 (AGPLv3 + Enterprise files + MIT packages + Application Exception; TRADEMARK.md). github.com/twentyhq/twenty (accessed 2026-11-04).

Yao, S. et al. (2022). ReAct. arXiv:2210.03629. | Schick, T. et al. (2023). Toolformer. arXiv:2302.04761.

Shinn, N. et al. (2023). Reflexion. arXiv:2303.11366. | Madaan, A. et al. (2023). Self-Refine. arXiv:2303.17651.

Park, J. et al. (2023). Generative Agents. arXiv:2304.03442. | Packer, C. et al. (2023). MemGPT. arXiv:2310.08560.

Wang, G. et al. (2023). Voyager. arXiv:2305.16291. | Jimenez, C. et al. (2023). SWE-bench. arXiv:2310.06770; Verified subset: OpenAI (2024-08), swebench.com.

Yang, J. et al. (2024). SWE-agent. arXiv:2405.15793. | Liu, X. et al. (2023). AgentBench. arXiv:2308.03688.

Mialon, G. et al. (2023). GAIA. arXiv:2311.12983; leaderboard: huggingface.co/spaces/gaia-benchmark/leaderboard (top 94.35%, Nov 2026).

Yuan, W. et al. (2024). Self-Rewarding Language Models. arXiv:2401.10020. | Zheng, W. et al. (2026). Do Agent Optimizers Compound? arXiv:2607.14004.

Novikov, A. et al. (2025). AlphaEvolve. arXiv:2506.13131. | Zhang, J. et al. (2025). Darwin Godel Machine. arXiv:2505.22954.

Zhang, K. et al. (2025). Agent Learning via Early Experience (Meta). arXiv:2510.08558. | Kwa, T. et al. (2025). Measuring AI Ability to Complete Long Tasks (METR). arXiv:2503.14499.

Xu, F. et al. (2024). TheAgentCompany. arXiv:2412.14161. | Merrill, J. et al. (2026). Terminal-Bench 2.0. arXiv:2601.11868; Li, Z. et al. (2026). LHTB 2.0. arXiv:2610.14146.

Tao, Z. et al. (2024). Survey on Self-Evolution of LLMs. arXiv:2404.14387. | Fang, J. et al. (2025). Survey of Self-Evolving AI Agents. arXiv:2508.07407.

Yao, Y. et al. (2025). Toward Super Agent System with Hybrid AI Routers. arXiv:2504.10519.

LangChain (2026). LangGraph 1.x docs: overview, persistence; customers (Klarna, Rippling, Vodafone). docs.langchain.com; langchain.com (accessed 2026-11-04).

OpenAI (2026). Agents SDK docs (MIT). openai.github.io/openai-agents-python. | Microsoft (2026). AutoGen archived -> agent-framework (MIT). github.com/microsoft/autogen, /microsoft/agent-framework.

browser-use (2026). README + docs (MIT; ~120k stars; stealth browsers $0.02/hr). github.com/browser-use/browser-use; docs.browser-use.com (accessed 2026-11-04).

Model Context Protocol (2026). Specification changelogs 2025-06-18 (OAuth 2.1) and 2026-07-28 (stateless); registry.modelcontextprotocol.io; AAIF donation (Dec 2025) and Agent Skills (Oct 2026) via Linux Foundation announcements.

Pricing (accessed 2026-11-04): tavily.com/pricing; serper.dev; brave.com/search/api; e2b.dev/pricing; modal.com/pricing; langfuse.com/pricing; clay.com/pricing; visualping.io/pricing; apollo.io/pricing.

Firecracker (2026). Documentation: <125 ms microVM start. github.com/firecracker-microvm/firecracker.

Security (2025-26): Brave, Indirect Prompt Injection remains fundamental (Jun 8); TechCrunch, OpenAI on AI-browser injection (Dec 22, 2025); Wiz, Agentic Browser Security year-in-review (Jan 16, 2026); Techdirt on Grok Bot Slack leak (Oct 19, 2026).

Funding (2025-26): SiliconANGLE - Clay $115M @ $7.1B (Sep 2026), Sierra $950M @ $15B (May 2026), Basis $100M @ $1.15B (Feb 2026); Forbes/Yahoo - Decagon $250M @ $4.5B (Jan-Feb 2026); TechCrunch - Cursor $2.3B @ $29.3B (Nov 2025), QA Wolf, Checkly, Momentic.

Stanford HAI (2025). AI Index Report 2025 - agentic benchmark gains. hai.stanford.edu/ai-index/2025-ai-index-report. | Gartner (2025-26). 40% agentic-AI cancellation projection, via Forbes (Jul 7, 2026) and MarTech (Apr 29, 2026).

OpenTelemetry (2026). GenAI semantic conventions - dedicated repository incl. MCP attributes. github.com/open-telemetry/semantic-conventions-genai.

swebench.com (2026). Leaderboard snapshot incl. top checked entry 74.4% (Claude 4.5 Opus, mini-SWE-agent) and Feb 2026 cohort; OpenAI's retirement note via HN item 47910388 (Apr 2026).